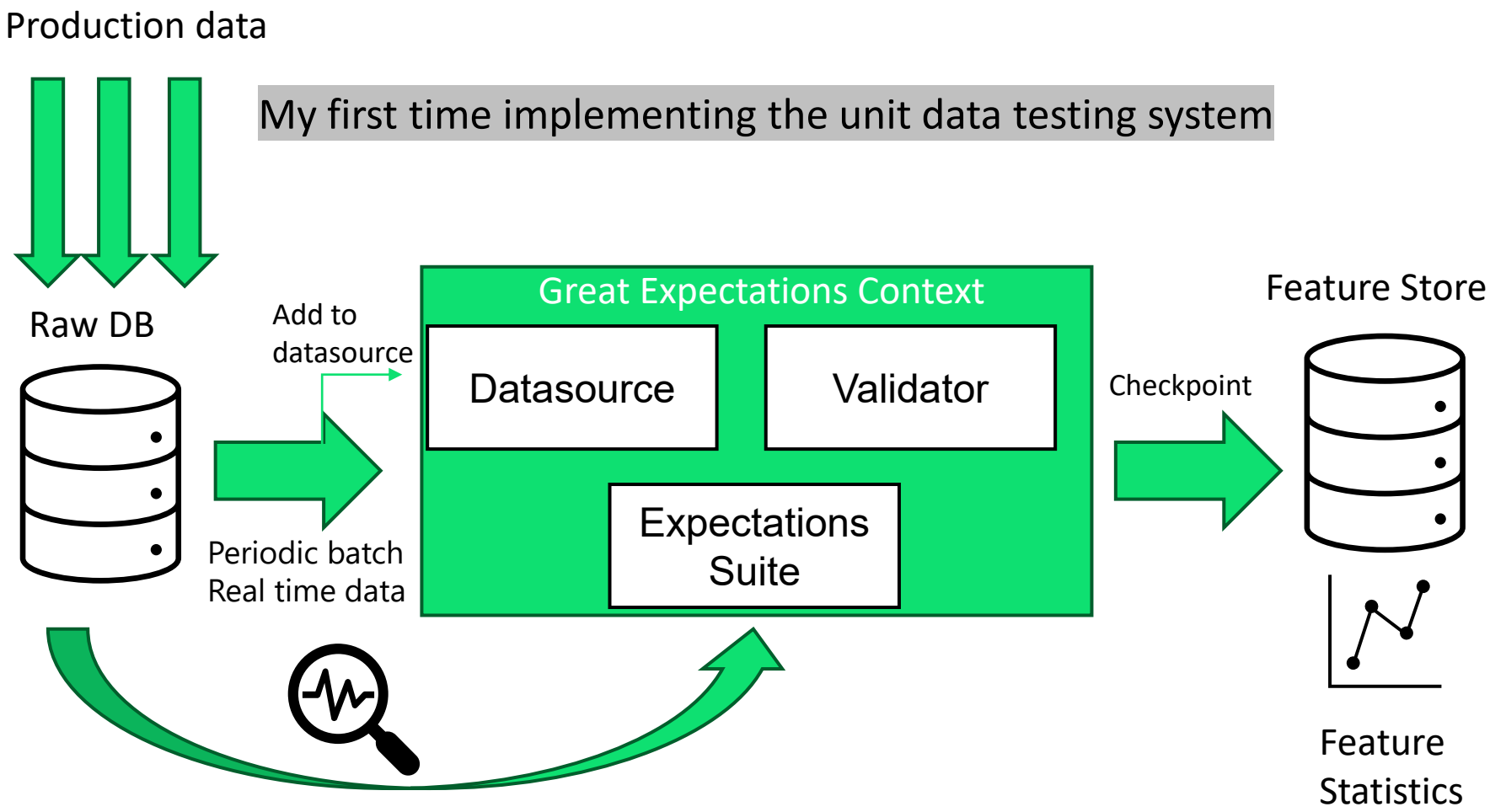
NOVA IMS INFORMATION MANAGEMENT SCHOOL

01: Model Development

Nuno Rosa
nagostinho@novaims.unl.pt

Last week recap

01: Model Development



Basis Expectations:

- Null Values
- Metrics based on Business (such as negative values)
- ...

01: Model Development

Production data

After some time gathering data

Raw DB

Add to datasource

Periodic batch
Real time data

Great Expectations Context

Datasource

Validator

Expectations Suite

Sequential checkpoints with business insights

Checkpoint 1

Checkpoint 2

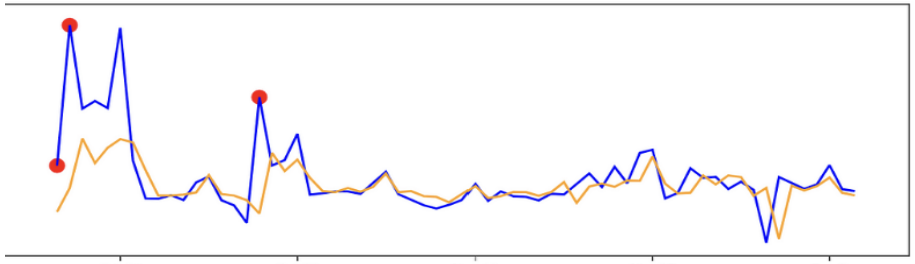
Checkpoint N

Feature Store

Reference data (e.g. last 3 months)

Industrialized expectations

Use feature statistics to build anomalies trend



01: Model Development

Transaction_Id	Sender_Id	Sender_Account	Sender_Country	Sender_Sector	Sender_Iob	Bene_Id	Bene_Account	Bene_Country	USD_Amount	label	Transaction_Type
PAY-BILL-3589	CLIENT-3566	ACCOUNT-3578	USA	21264	CCB	COMPANY-3574	ACCOUNT-3587	GERMANY	492.67	0	MAKE-PAYMENT
WITHDRAWAL-3591	CLIENT-3566	ACCOUNT-3579	USA	18885	CCB				388.92	0	WITHDRAWAL
MOVE-FUNDS-3528	CLIENT-3508	ACCOUNT-3520	USA	4809	CCB	COMPANY-3516	ACCOUNT-3527	GERMANY	280.7	0	MOVE-FUNDS
WITHDRAWAL-3529	CLIENT-3508	ACCOUNT-3519	USA	7455	CCB				118.14	0	WITHDRAWAL
QUICK-DEPOSIT-3471						CLIENT-3442	ACCOUNT-3461	USA	105.16	0	DEPOSIT-CASH
QUICK-DEPOSIT-3473						CLIENT-3442	ACCOUNT-3460	USA	164.97	0	DEPOSIT-CASH
PAY-BILL-3404	CLIENT-3384	ACCOUNT-3395	USA	36316	CCB	COMPANY-3392	ACCOUNT-3401	GERMANY	456.89	0	MAKE-PAYMENT
QUICK-DEPOSIT-3406						CLIENT-3384	ACCOUNT-3396	USA	413.17	0	DEPOSIT-CASH
PAY-CHECK-3347	CLIENT-3330	ACCOUNT-3341	USA	36194	CCB	CLIENT-3333	ACCOUNT-3338	CANADA	377.65	0	PAY-CHECK
PAY-CHECK-3348	CLIENT-3330	ACCOUNT-3340	USA	20626	CCB	CLIENT-3333	ACCOUNT-3338	CANADA	338.03	0	PAY-CHECK
MOVE-FUNDS-3292	CLIENT-3272	ACCOUNT-3284	USA	21568	CCB	CLIENT-3275	ACCOUNT-3291	CANADA	100.85	0	MOVE-FUNDS
MOVE-FUNDS-3294	CLIENT-3272	ACCOUNT-3284	USA	29040	CCB	CLIENT-3273	ACCOUNT-3289	USA	276.66	0	MOVE-FUNDS
PAY-BILL-3232	CLIENT-3203	ACCOUNT-3222	USA	27393	CCB	COMPANY-3210	ACCOUNT-3218	GERMANY	234.88	0	MAKE-PAYMENT
QUICK-DEPOSIT-3234						CLIENT-3203	ACCOUNT-3222	USA	945.22	0	DEPOSIT-CASH
DEPOSIT-CASH-3163						CLIENT-3139	ACCOUNT-3154	USA	655.09	0	DEPOSIT-CASH
PAY-BILL-3162	CLIENT-3139	ACCOUNT-3153	USA	25066	CCB	COMPANY-3147	ACCOUNT-3160	GERMANY	675.37	0	MAKE-PAYMENT
WITHDRAWAL-3100	CLIENT-3075	ACCOUNT-3090	USA	22778	CCB				319.95	0	EXCHANGE
QUICK-PAYMENT-3099	CLIENT-3075	ACCOUNT-3091	USA	39013	CCB	CLIENT-3078	ACCOUNT-3087	TAIWAN	771.54	0	QUICK-PAYMENT
PAY-BILL-3036	CLIENT-3016	ACCOUNT-3028	USA	43951	CCB	COMPANY-3022	ACCOUNT-3033	GERMANY	730.69	0	MAKE-PAYMENT

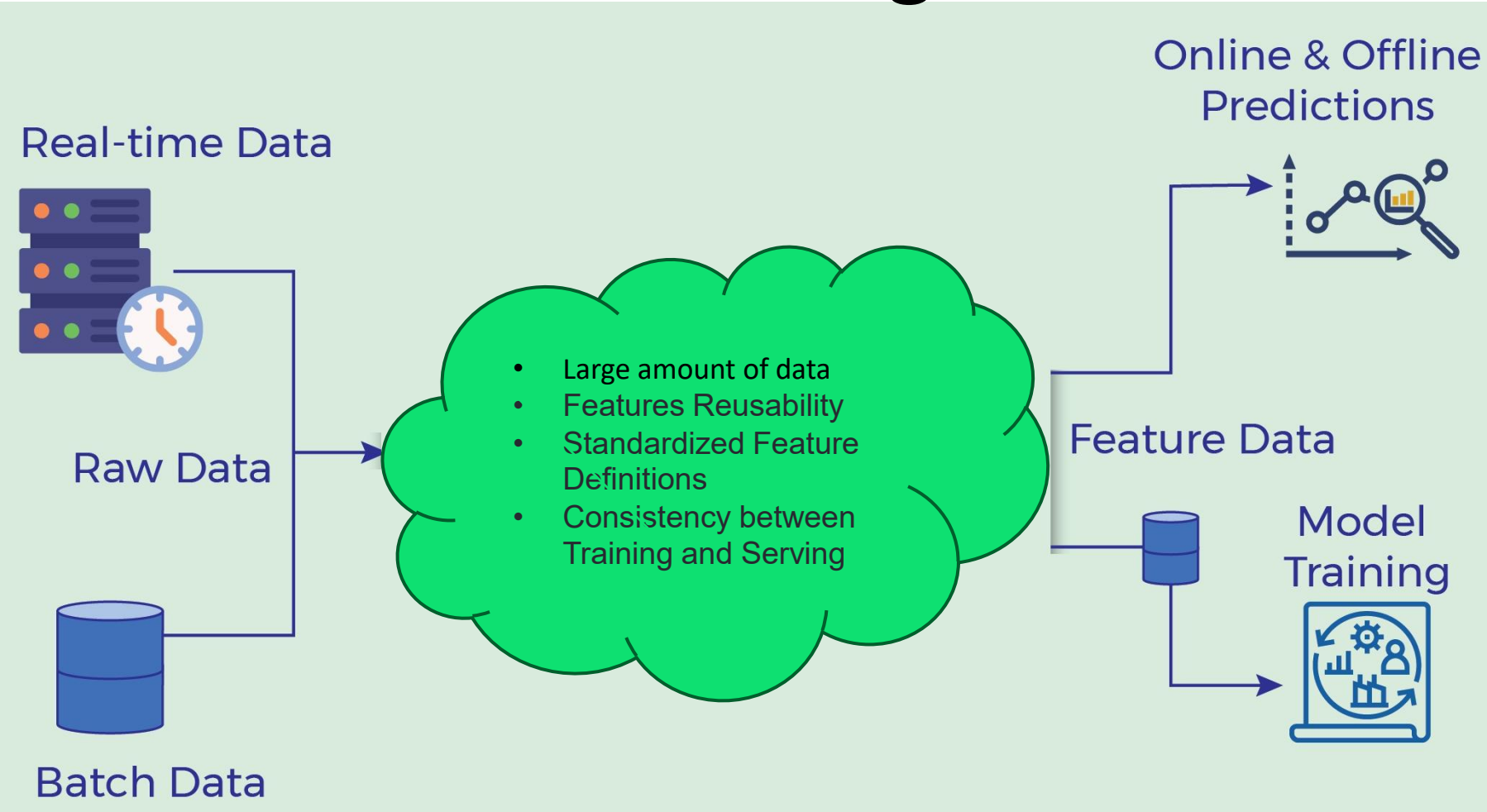
Possible business expectations:

- When the send_id is null the type of operation is always deposit-cash?
- When there is withdrawal, there is no beneficiary account.
- Percentage of sender account filling over the time vs beneficiary account (can reveal processing problems on the system).

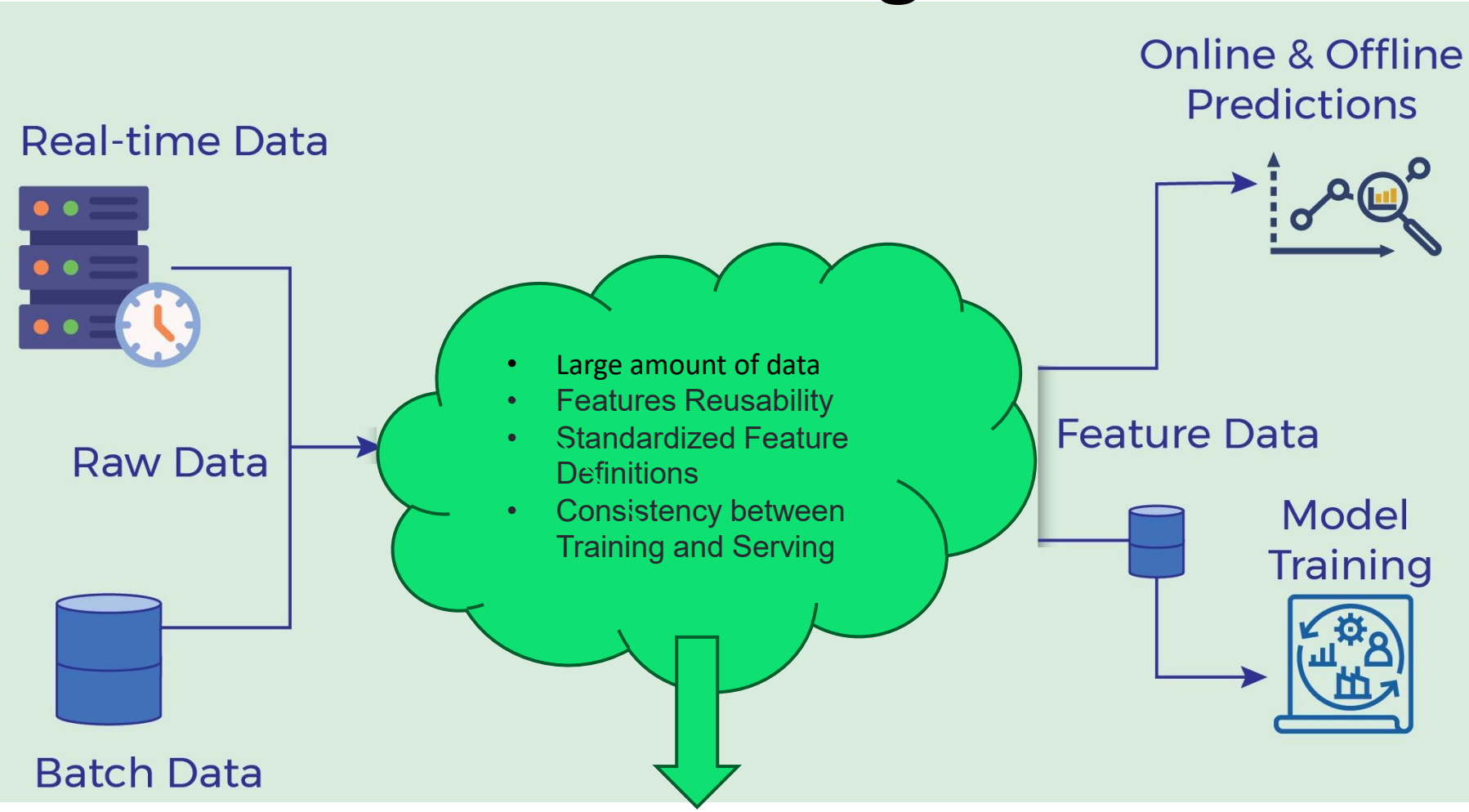
Model Development

Feature management

01: Model Development
Feature challenges



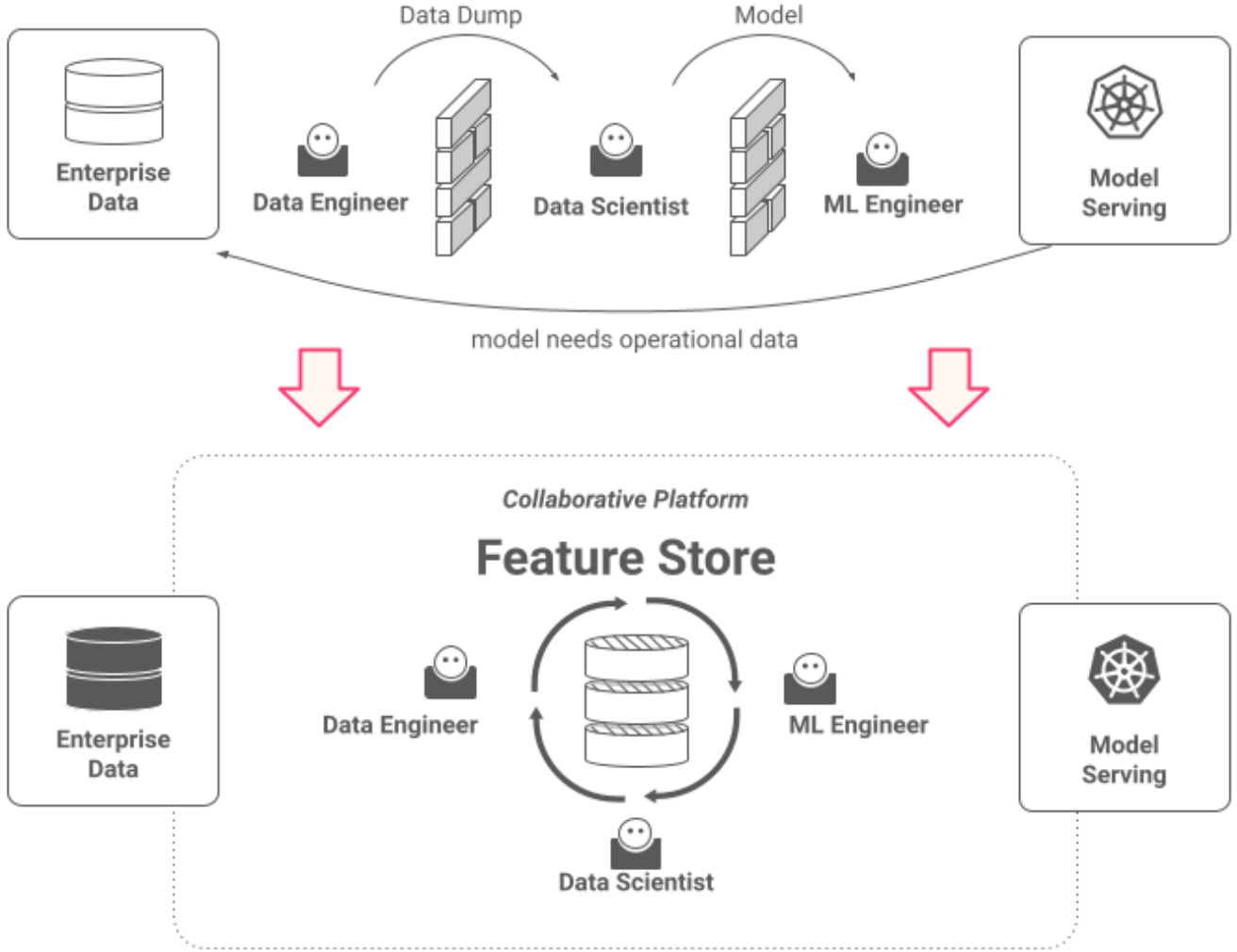
01: Model Development
Feature challenges



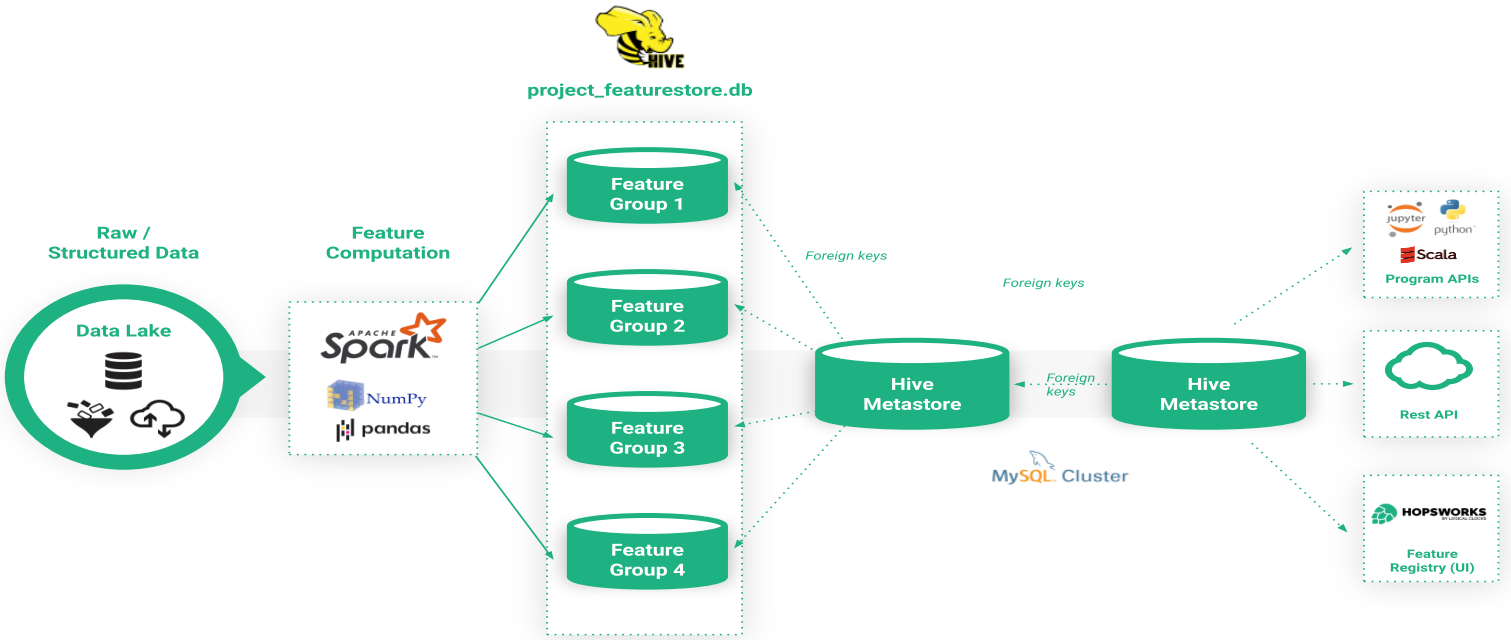
The Feature Store
A Hub for Collaboration, Consistency, Centralized Management and Accessibility of Features

01: Model Development
Feature Stores

- **Feature stores are repositories of different features** associated with business entities that are created and stored in a central location for easier reuse.
- They usually **combine an offline part** (slower, but potentially more powerful) and an **online part** (quicker and more useful for real-time needs).



01: Model Development
Feature Stores



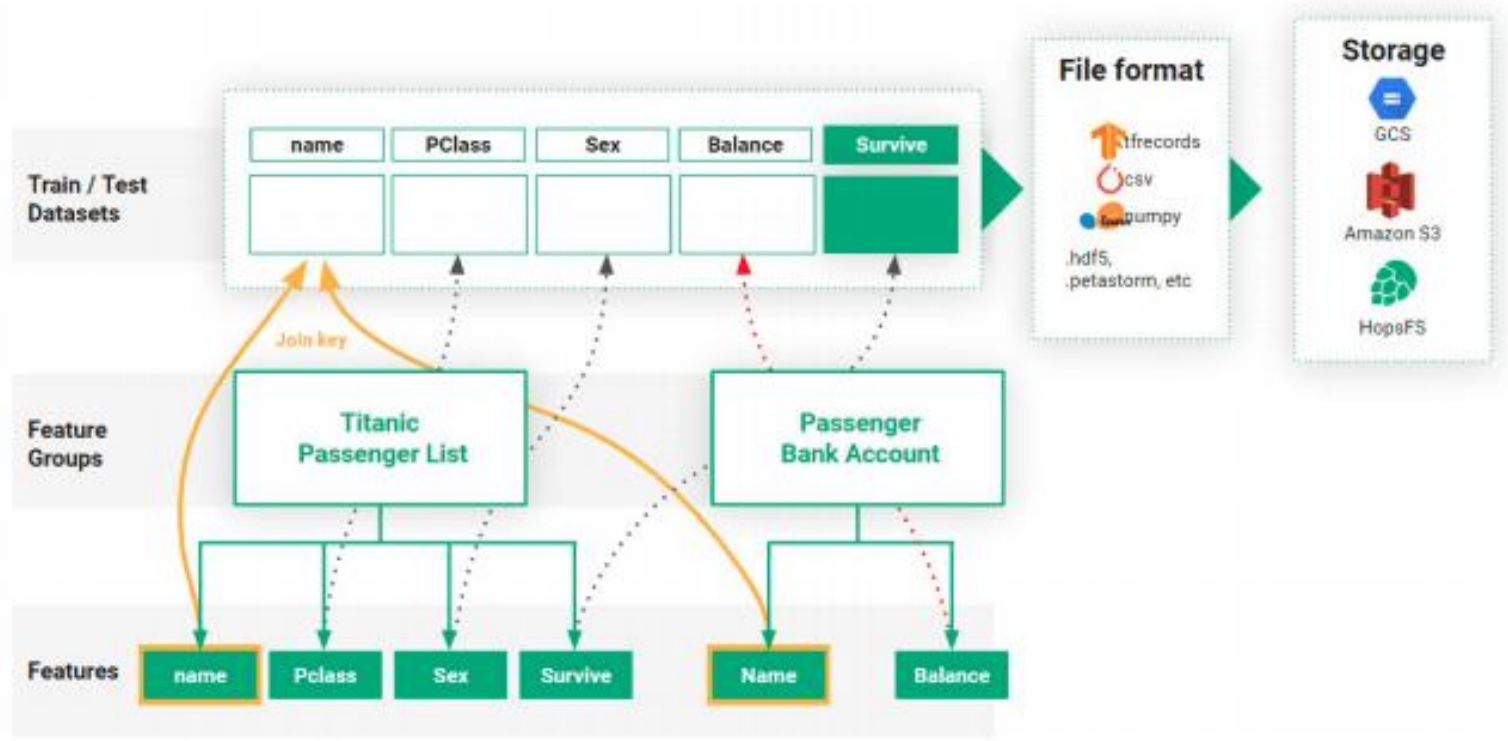
Challenges

1. Amount of data
2. Creating features from raw data (ineffective, non-standard and expensive feature engineering)
3. Training the model
4. Making predictions (inference) on new data. In large organizations, these phases can span different teams

Advantages

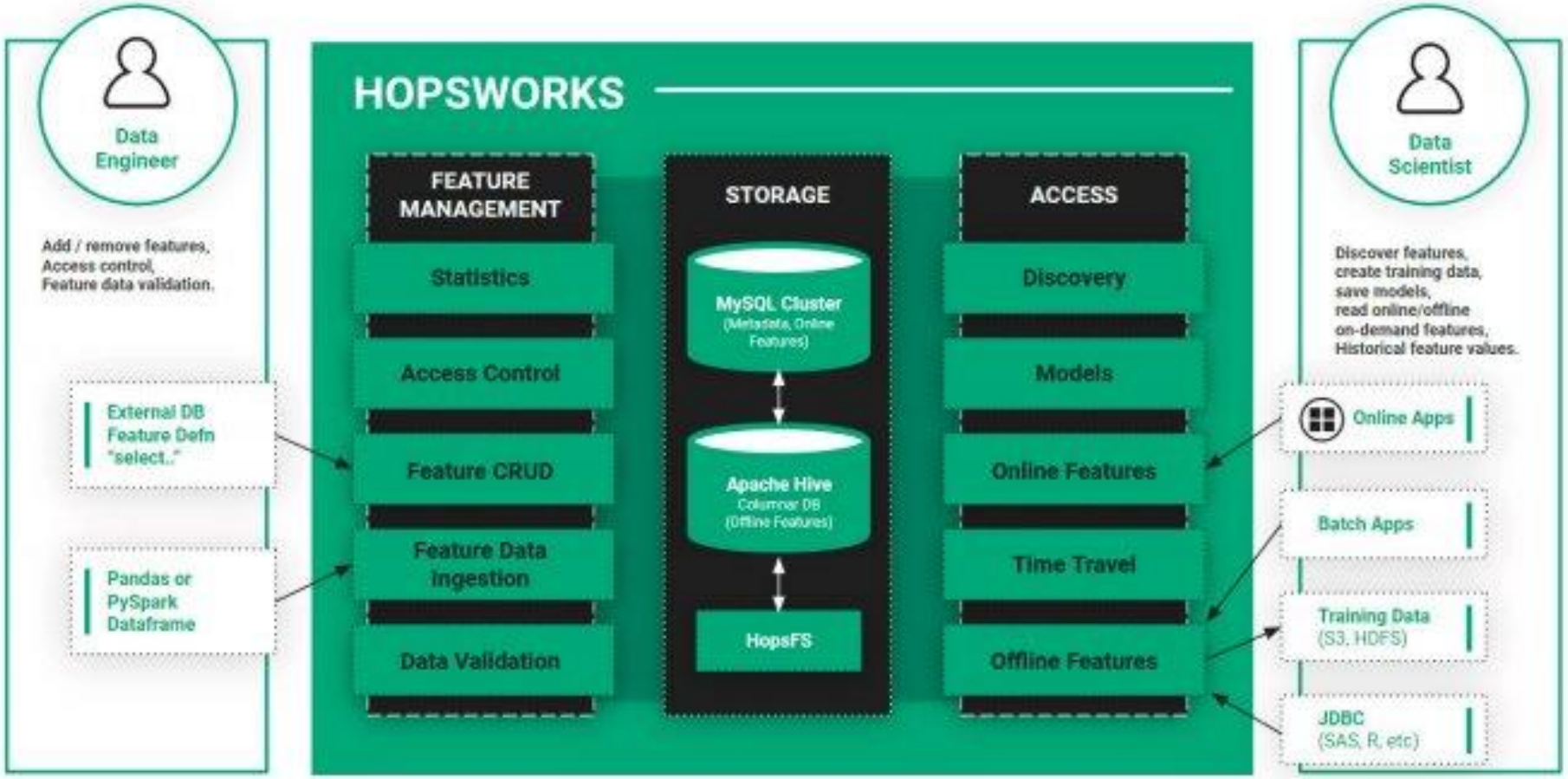
1. A data warehouse for machine learning features: the discovery, monitoring, analysis, and reuse of features within an organization.
2. Management, access and creation of point-in-time datasets from historical feature data.
3. An operational store for precomputed features that online models can use to augment their feature vectors with features computed from historical data and context data.

01: Model Development
Feature Stores

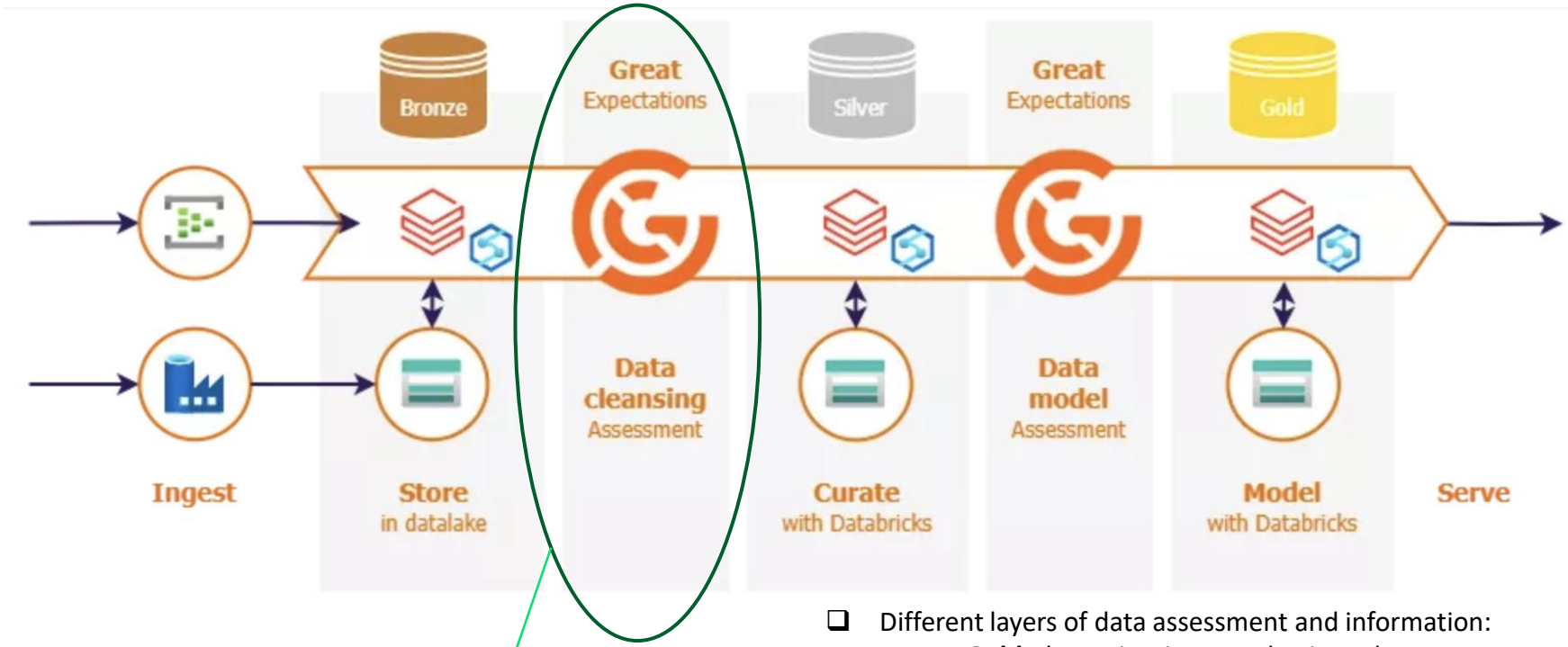


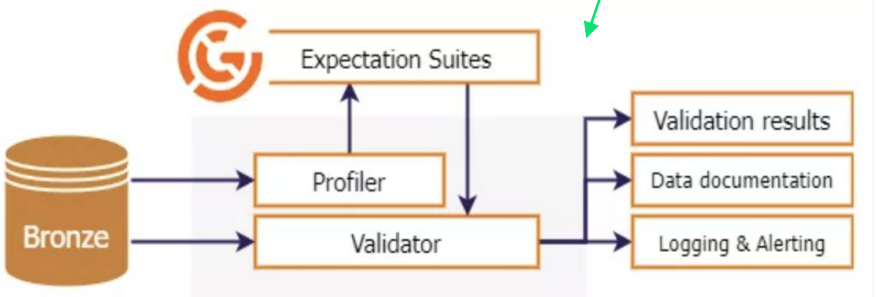
- Each feature belongs to a Feature Group with an associated key for computation.
- Data Scientists can generate training and testing data by providing/selecting a set of features, the target file format for the output of features
- For **real-time applications** such as fraud detection or dynamic pricing, the ability of a feature store to rapidly process and serve up-to-date features is crucial. It ensures that live decision-making models receive the most relevant, current data, directly impacting their effectiveness.

01: Model Development
Feature Stores



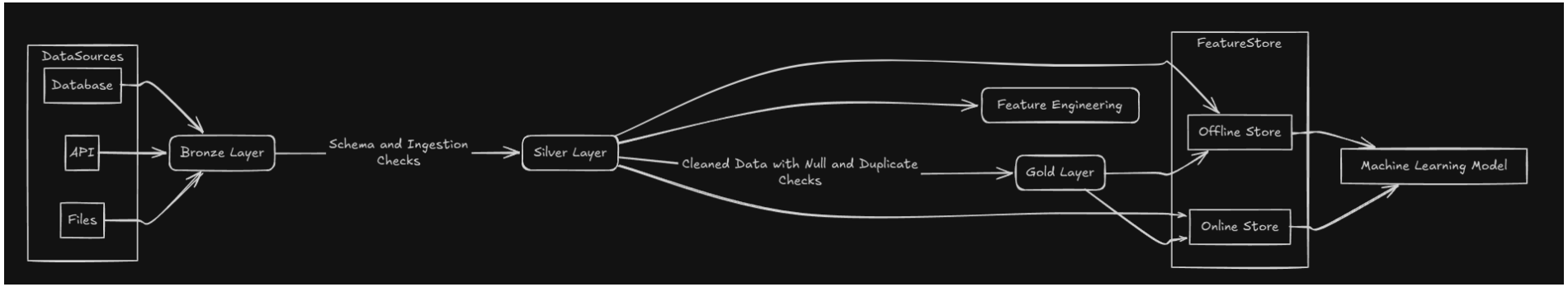
01: Model Development
Our goal





- Different layers of data assessment and information:
  - **Gold**: data scientists, ML, business layers
  - **Silver**: data engineers for feature engineering
  - **Bronze**: data ingestion
- Two GX layers:
  - **General**: are amounts of the correct numeric data type? Are numbers in the correct format? Are all values of a mandatory column filled in? Are column names and values consistent? Are key-column values unique?
  - **Focused**: Are there outliers for a certain measure? Are these amounts in realistic ranges and standard deviations?

01: Model Development
Feature Store: Medallion Archtecture



The **Feature Store** sits alongside the Gold layer and integrates with both Silver and Gold:

- **Offline Store**:
  - ❑ Reads from **Silver or Gold tables** for batch processing and historical training data.
  - ❑ Features are versioned and stored with timestamps and entity IDs.

- **Online Store**:
  - ❑ Serves low-latency, real-time features to ML models in production.
  - ❑ Features are ingested from streaming sources or materialized views.

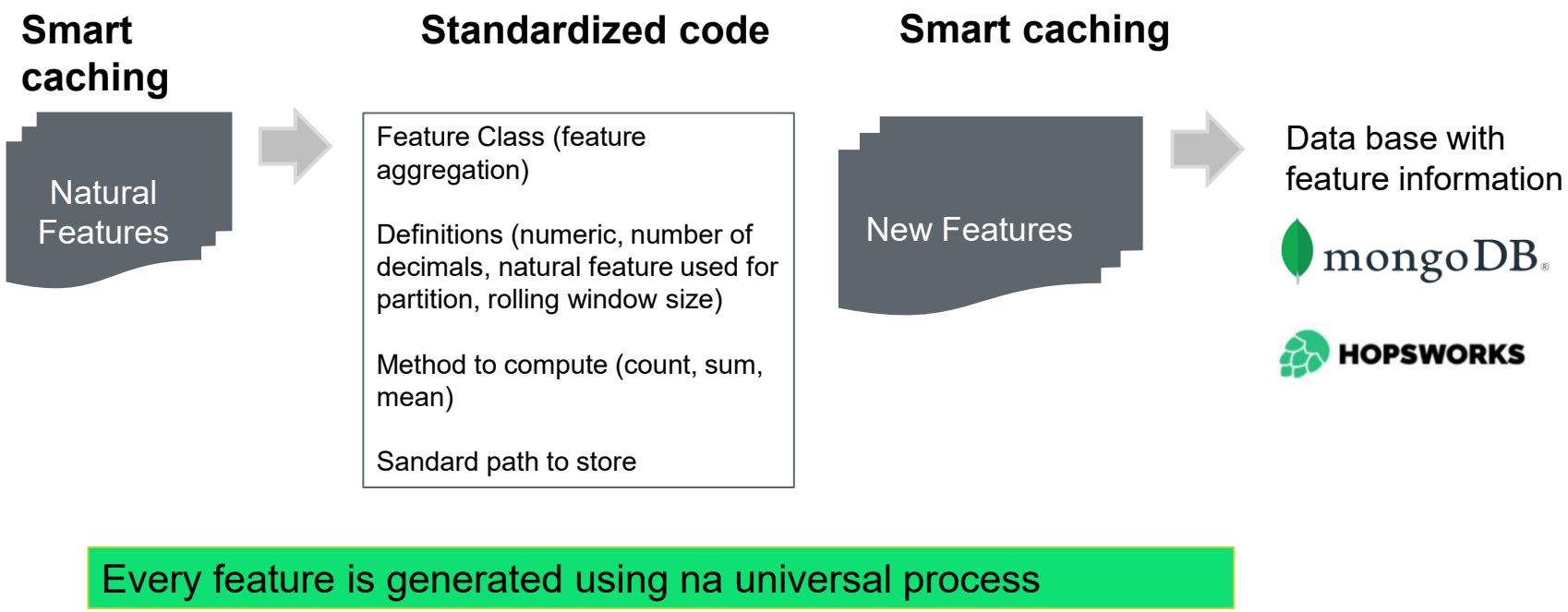
- **Feature Definitions**:
  - ❑ Stored centrally with metadata, lineage, and freshness info.
  - ❑ Reused across training and serving to ensure **training-serving skew** is avoided.

01: Model Development
Feature Management

Features can be classes with attributes and methods, as a "a first-class citizen". Each method and attribute will provide useful information for the entire team:

- Source – Where the data comes from (table or method of computation).
- Name – The name of the feature on the data source (e.g. table column), with a well-defined nomenclature.
- Type – Int, Float, String allows for enforcing the correct behavior in filters and transformations.
- Description and comments added by the Data Scientists to keep track of insights about the feature, meaning of certain values (or where to find it). The impacts on the models, importance and metric summaries.
- Some statistical measures about the features (cardinality, range).

01: Model Development
Feature Management



Every feature is generated using na universal process

01: Model Development
Feature Management

```
    'CLIENT_ID':                                    [ StringF, {}],
    'OPERATION_AMOUNT':                             [ NumericF, {}],
    'C_CLIENTID_TF_90D_1D_TOT_AMOUNT':              [ AggF, {
                                                        'index': Feature('CLIENT_ID'),
                                                        'index_conditions': {'CLIENT_ID': {'$ne': 'Ø'}},
                                                        'evaluation_conditions': {'TRANSACTIONTYPE': {'$eq': 'SERVICES_PAYMENTS'}},
                                                        'reference_attribute': 'OPERATION_AMOUNT',
                                                        'metric': CounterMetrics.TOTAL_AMOUNT,
                                                        'decimals': 0,
                                                        'precision_time': 1,
                                                        'include_current': False,
                                                        'time_range_min': 1,
                                                        'time_range_max': 90,
                                                        'period_unit': CounterPeriodUnits.DAYS,
                                                    }],
```

- Create a dictionary telling what type of feature is: if it is a natural feature of the event, or a result of some aggregation.
- Create a dictionary with the recipe of how a feature should be calculated.

01: Model Development
Feature Management

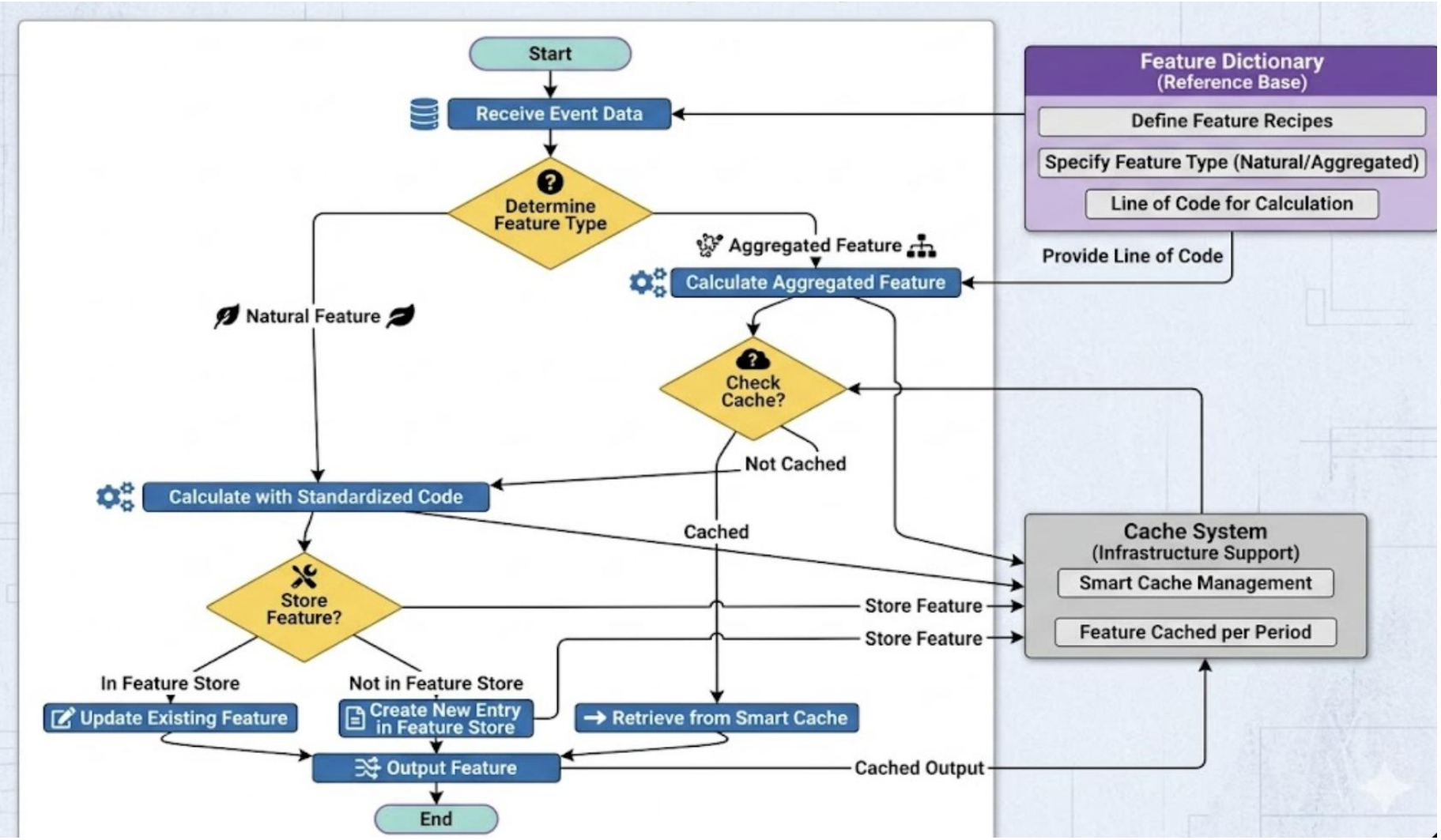
```python
class NaturalFeature(ABC):

    def __init__(self, feature_name: str, *,
                 partition_by_column, id_column):

        self._feature_name = feature_name
        self._partition_by_column = partition_by_column
        self._id_column = id_column
        self._is_category = None
        self._is_numeric = None


    def get_raw_name(self, *, feature_name: str = None, inserted_year_month: int = None

    def get_cache_name(self, inserted_year_month: int, updated_date: int = 0,
                       tag_usecase: str = "", feature_name: str = None):

    def get_cache_path(self, inserted_year_month: int, updated_date: int = 0,
                       tag_usecase: str = "", feature_name: str = None):

    def get_name(self):

    def is_category(self):

    def is_numeric(self):

    def _get_info(self):

    def _get_current_user(self):

    def _save_info(self, doc:dict, changes:dict):

    def get_comments(self):

    def add_comments(self, comments:Union[list,str], *, replace_existing=False):
```

```python
dataframe = get_feature(AggF("C_CLIENTID_TF_90D_1D_TOT_AMOUNT"), date=202304)
```

```python
class BaseSinteticFeature(NaturalFeature):

    def __init__(self, feature_name:str, partition_by_column: str='internal_odi_cde', id_column: str="id"):

        self.
        self.
        self._is_numeric = None
        self._id_column = id_column
        self._partition_by_column = partition_by_column


    def get_raw_name(self):

    def is_sintetic(self):

    @abstractmethod
    def get_cache_name(self, inserted_year_month:int, updated_year_month:int=0, tag_usecase:str=""):
        pass

class AggF(SinteticFeature):

    def get_base_features(self):


    def get_cache_name(self, inserted_year_month: int, updated_date: int = 0, tag_usecase:str=""):


    def get_cache_path(self, inserted_year_month: int, updated_date: int=0, tag_usecase:str=""):


    def compute_cache(self, spark: SparkSession, inserted_date: int):


    def create_cache(self, spark: SparkSession, inserted_date: int,
                     updated_date):
```

(variable) _is_numeric: None

- The idea is just with one line create the feature using all the define methods in the class



The Feature Execution Pipeline

- **Ingestion:** System receives event & queries the Feature Dictionary to resolve the taxonomy (Natural vs. Aggregated).
- **Smart Caching:** For aggregations, check the Feature Store cache first. A cache hit bypasses expensive compute, minimizing latency.
- **Standardized Compute:** On a cache miss, dynamically generate code using OOP recipes (Configuration over Code) to prevent training-serving skew.
- **Persistence:** Write the newly calculated feature back to the Store and Cache.
- **Output:** Serve the feature vector to the downstream ML model seamlessly.

01: Model Development
Feature Engineering

There are several strategies to create new features:

- **Old fashioned ones**

- **Automated ones**

tsfresh

Time series, feature synthesis, feature selection

Featuretools

Relational tables, feature synthesis, feature selection

Feature engineering category	Description
Derivatives	Infer new information from existing information—e.g., what day of the week is this date?
Enrichment	Add new external information—e.g., is this day a public holiday?
Encoding	Present the same information differently—e.g., day of the week or weekday versus weekend.
Combination	Link features together—e.g., the size of the backlog might need to be weighted by the complexity of the different items in it.

How Feature Selection Impacts MLOps Strategy?

- The model can become more and more expensive to compute.
- More features require more inputs and more maintenance down the line.
- More features mean a loss of some stability.
- The sheer number of features can raise privacy concerns.

01: Model Development
Feature Tools

01.EntitySet

- The foundation of Featuretools. It is the container of our data.
- Represents your dataset as a collection of related tables/entities.
- Contains information about the tables, their relationships, indexes, etc.

02. Relationships

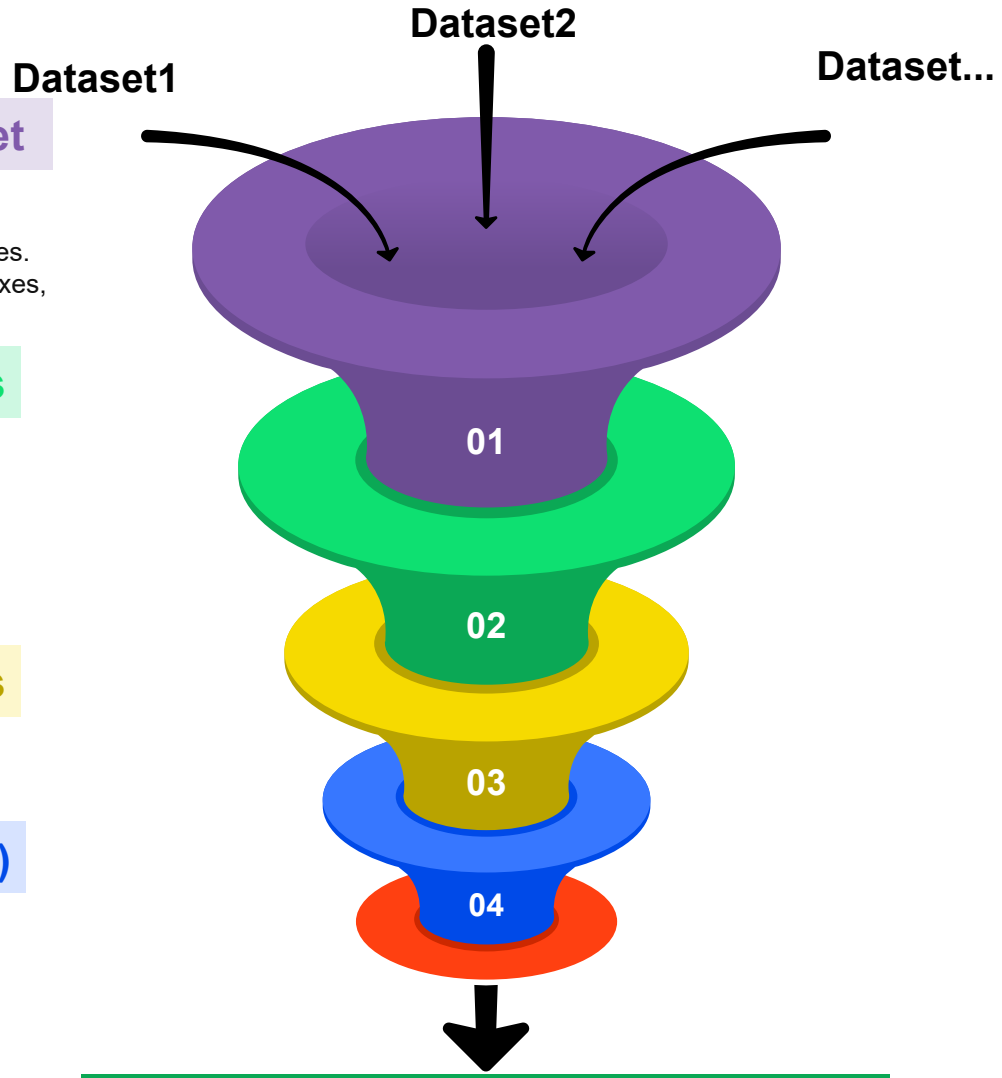
- Establishing relationships between the entities is a fundamental concept of feature tools.
- The most common type of relationship is one-to-many (a parent is a single individual, but can have mutliple children).

03. Feature Primitives

- An operation applied to a data frame for creating new features (transformations and aggregations).

04. Deep Feature Synthesis (DFS)

- Generates new features by combining existing ones.
- Creates features based on aggregation, transformation, and other operations.
- DFS explores relationships across tables and time to create meaningful features.



- A table where each row corresponds to an instance, and the columns represents the engineered features.

01: Model Development
Feature Tools

01.EntitySet

- The foundation of Featuretools. It is the container of our data.
- Represents your dataset as a collection of related tables/entities.
- Contains information about the tables, their relationships, indexes, etc.

02. Relationships

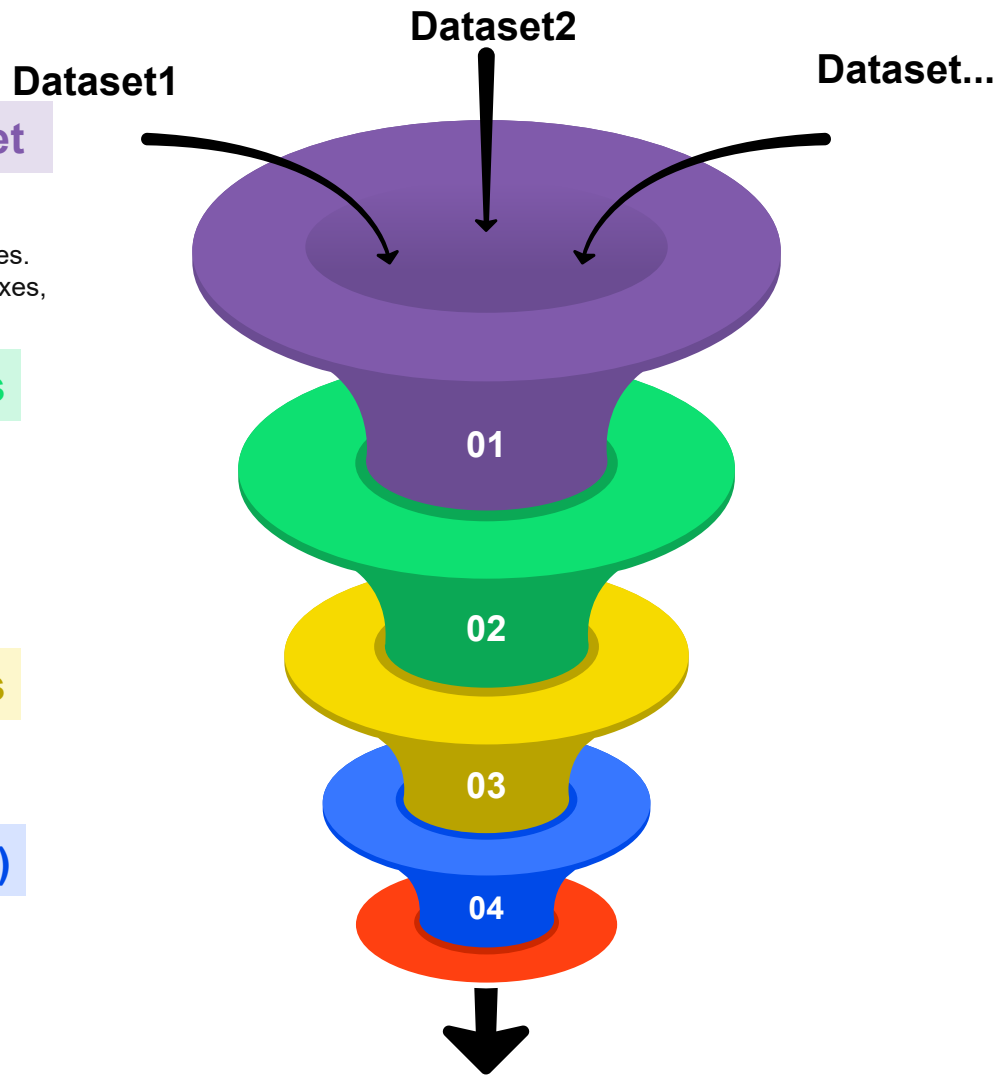
- Establishing relationships between the entities is a fundamental concept of feature tools.
- The most common type of relationship is one-to-many (a parent is a single individual, but can have mutliple children).

03. Feature Primitives

- An operation applied to a data frame for creating new features (transformations and aggregations).

04. Deep Feature Synthesis (DFS)

- Generates new features by combining existing ones.
- Creates features based on aggregation, transformation, and other operations.
- DFS explores relationships across tables and time to create meaningful features.



CRITICAL THINKING: ANALYSE YOUR OWN DATA TO CREATE THE MOST RELEVANT FEATURES (NO DEPENDENCY ON AUTOMATED TOOLS)

Model Development

Experimentation

01: Model Development

Training data

- Training data is usually labeled for the prediction case with examples of what is being modeled.
- Should be an accurate projection of the reality. Sampling it in the proper way can be crucial to obtain the best results capable of generalizing the model.

Performance metric

- Performance metrics and business goals should be fully married on this honeymoon. The model should be for meeting the business goals.
- In a churn example, if a bank contacts all clients that the model predicts as possible cases of churn, that it should be exist a sweet point between the "saved" clients and the costs of the call center.

ML algorithm

- There are a variety of models that work in various ways and have different pros and cons.
- It is important to note that some algorithms are more suited to certain tasks than others, but their selection also depends on what needs to be prioritized: performance, stability, interpretability, computation cost.

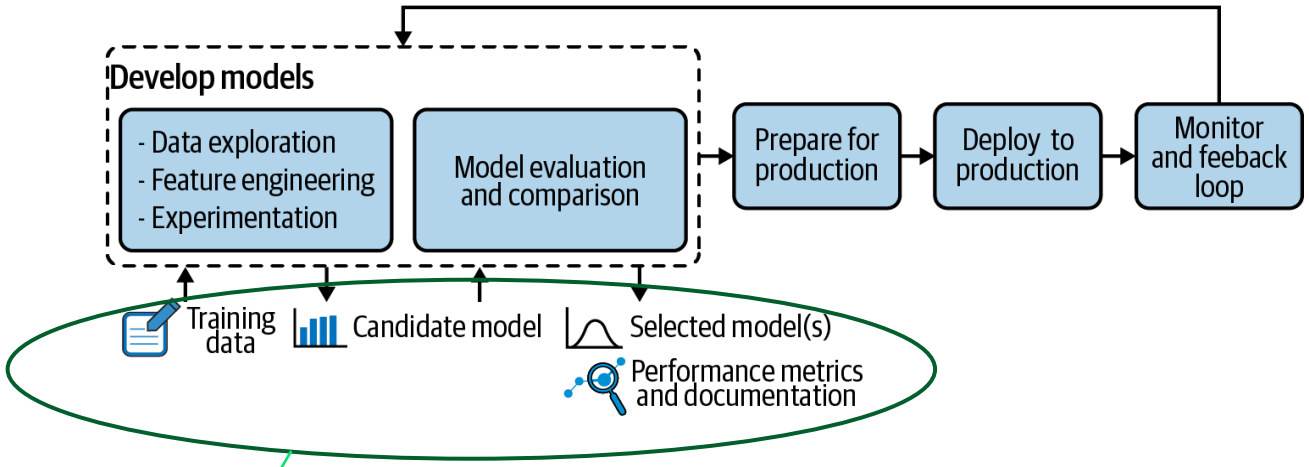
Hyperparameters

- The algorithm contains the basic formula, the parameters it learns are the operations and operands that make up this formula for that particular prediction task, and the hyperparameters are the ways that the algorithm may go to find these parameters. For example, in a decision tree (where data continues to be split in two according to what looks to be the best predictor in the subset that reached this path), one hyperparameter is the depth of the tree (i.e., the number of splits).

Evaluation dataset

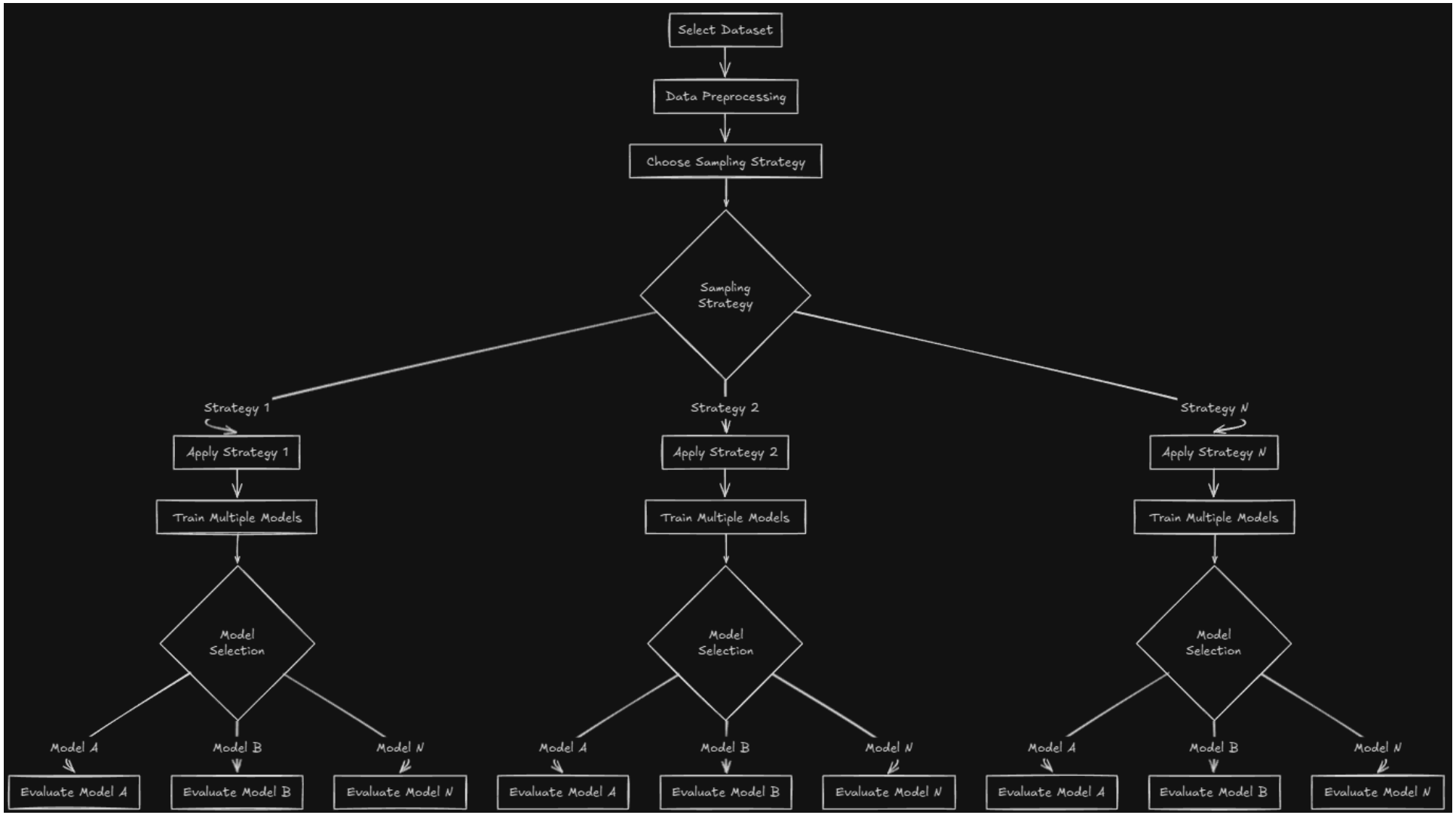
- When using labeled data, an evaluation dataset that is different from the training set will be required to evaluate how the model performs on unseen data (i.e., how well it can generalize).

01: Model Development

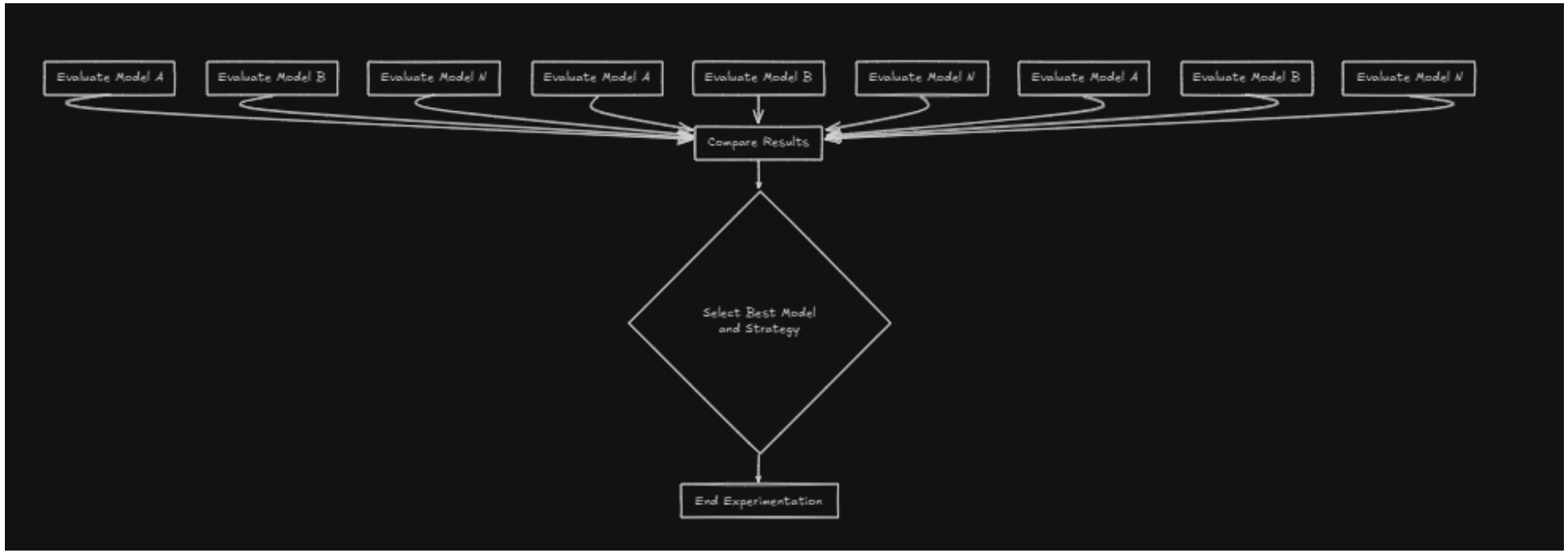


- **A data scientist will constantly experiment** with new datasets, models, software libraries, business metrics.
- Jupyter notebooks are great for experimentation, but **we need to track experimentation**.

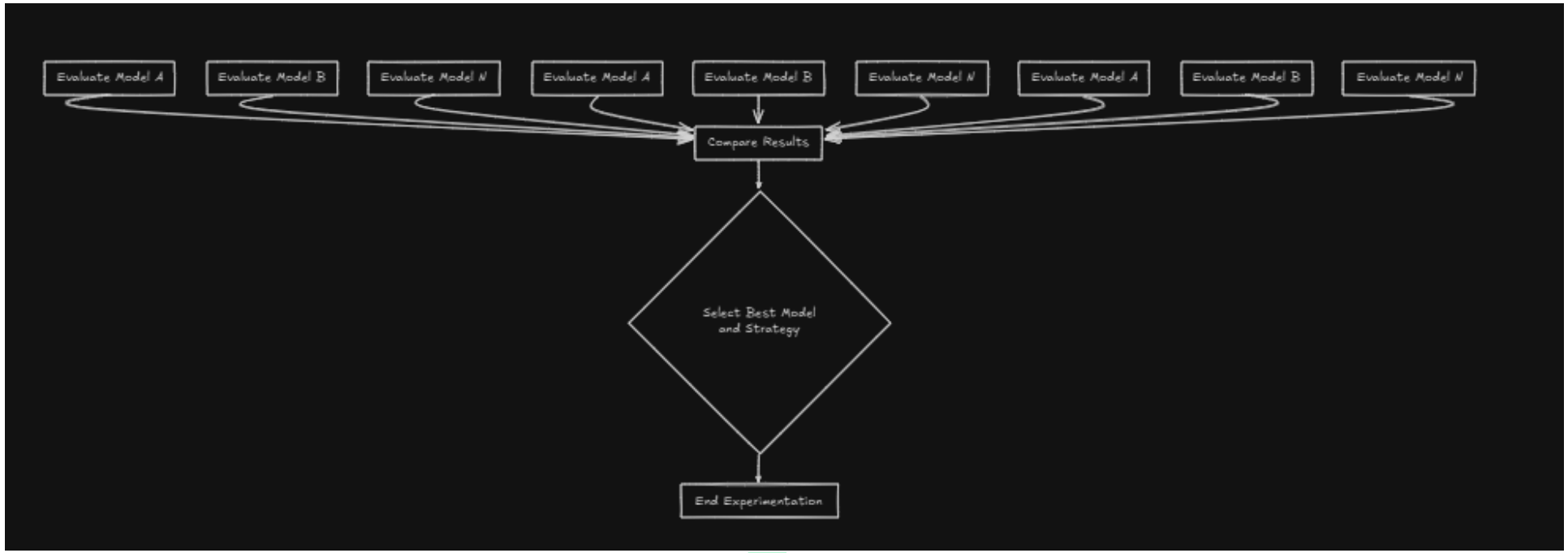
01: Model Development

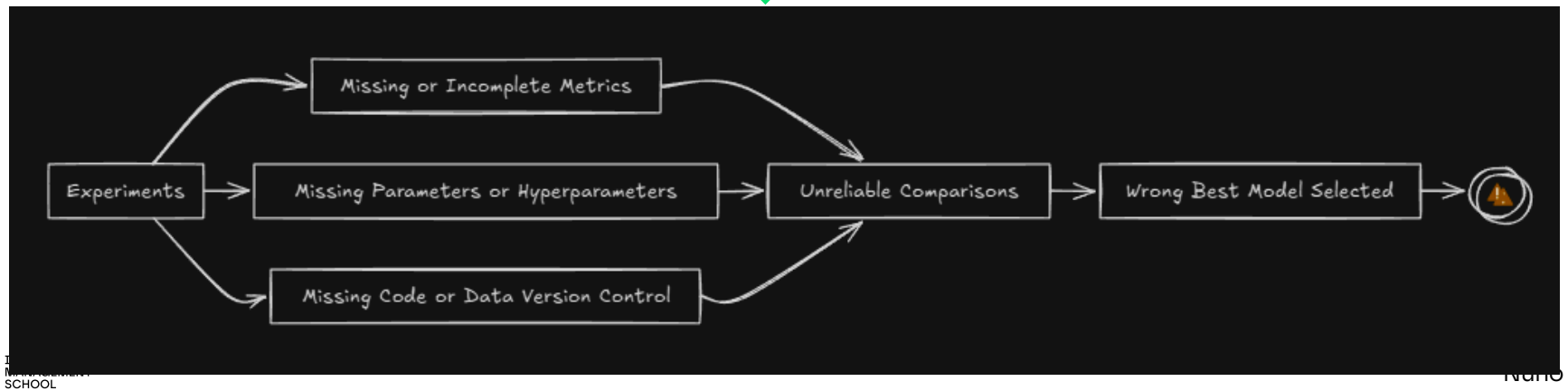


01: Model Development

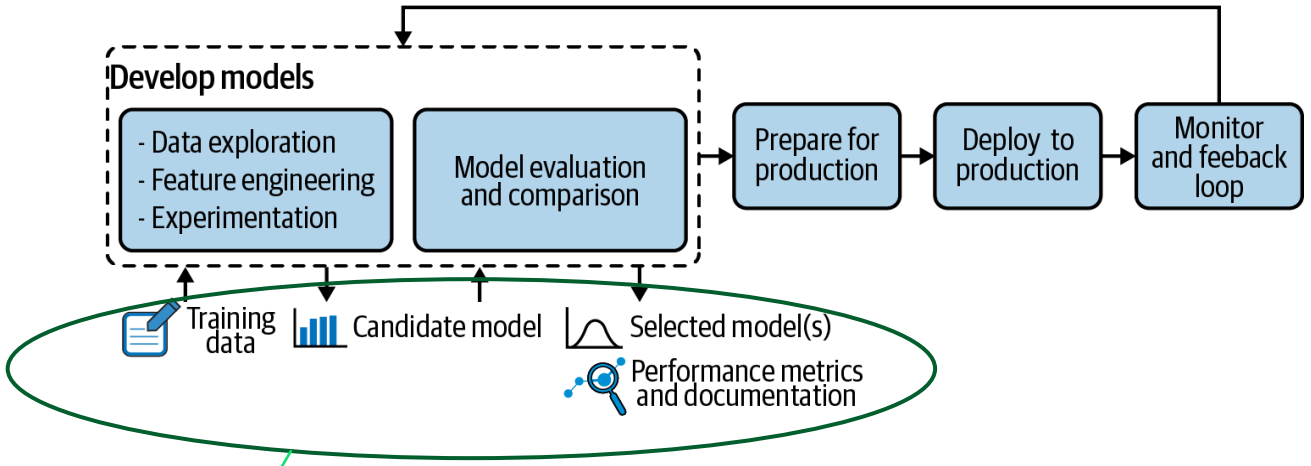


01: Model Development





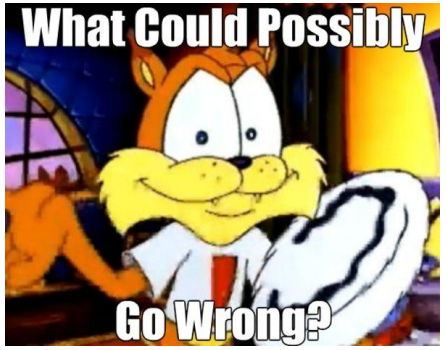
01: Model Development



- **A data scientist will constantly experiment** with new datasets, models, software libraries, business metrics.
- Jupyter notebooks are great for experimentation, but **we need to track experimentation**.

Our usual working folder...

```
-rw-rw-r-- 1 m                    :h 5097 Dez 20 20:04 Untitled1.ipynb
-rw-rw-r-- 1 m                    :h  715 Jan  7 19:18 Untitled2.ipynb
-rw-rw-r-- 1 m                    :h   72 Dez 20 19:34 Untitled3.ipynb
-rw-rw-r-- 1 m                    :h   72 Dez 28 11:48 Untitled4.ipynb
-rw-rw-r-- 1 m                    :h 3505 Jan  7 19:19 Untitled5.ipynb
-rw-rw-r-- 1 m                    :h   72 Dez 20 12:38 Untitled.ipynb
```



01: Model Development

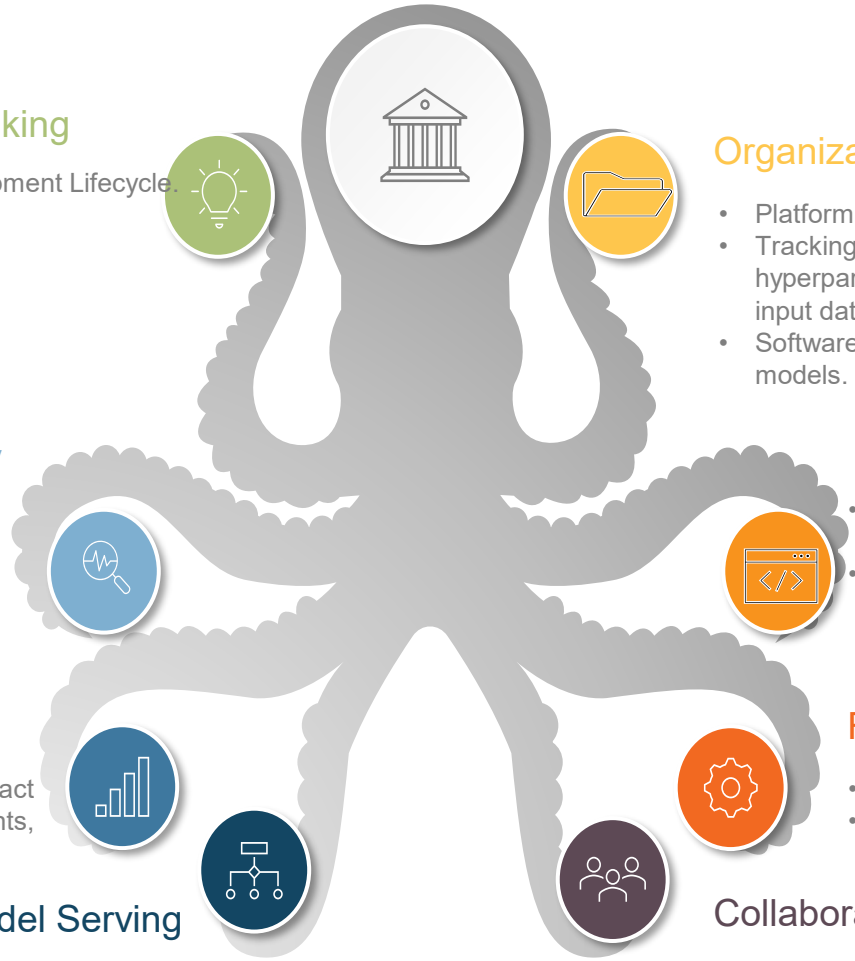
```python
# Load libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
import joblib

# Load data
df = pd.read_csv("churn.csv")

# Preprocessing manually
df = df.dropna()
X = df.drop(columns=["customerID", "Churn"])
y = df["Churn"].map({"Yes": 1, "No": 0})

# Split
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

# Train model (random hyperparameters)
model = RandomForestClassifier(n_estimators=200, max_depth=7)
model.fit(X_train, y_train)

# Evaluate
print("Train accuracy:", model.score(X_train, y_train))
print("Test accuracy:", model.score(X_test, y_test))

# Save model manually
joblib.dump(model, "churn_model.pkl")
```

- No **version control** of models or data.
- **No recording of hyperparameters.**
- Data preprocessing **not reproducible or logged.**
- No separation between training/testing preprocessing.
- **No metrics logged** systematically.
- **No reproducibility** across experiments.
- **Manual model saving**, churn_model.pkl will be overwritten with no history.

01: Model Development

01: Model Development

Success ingredients



Decision Making

- Platform to unify ML development Lifecycle.

Organization

- Platform to unify ML development Lifecycle.
- Tracking logs parameters, centralized hyperparameters versions, keep track of the input data.
- Software requirements used to create the models.

Model Registry

- Management of the different stages, model versions, aliasing.
- Smooth productionisation.

Integration with Popular Tools

- Python: most used programming language in Machine Learning.
- Easy to integrate in exploring Jupyter Notebooks.

UI

- User-friendly UI to interact with the tracked experiments, compare runs and model versions.

Reproducibility and Auditing

- Integration with CI/CD pipelines.
- Capturing the environment (dependencies, libraries, and system configurations)

Model Serving

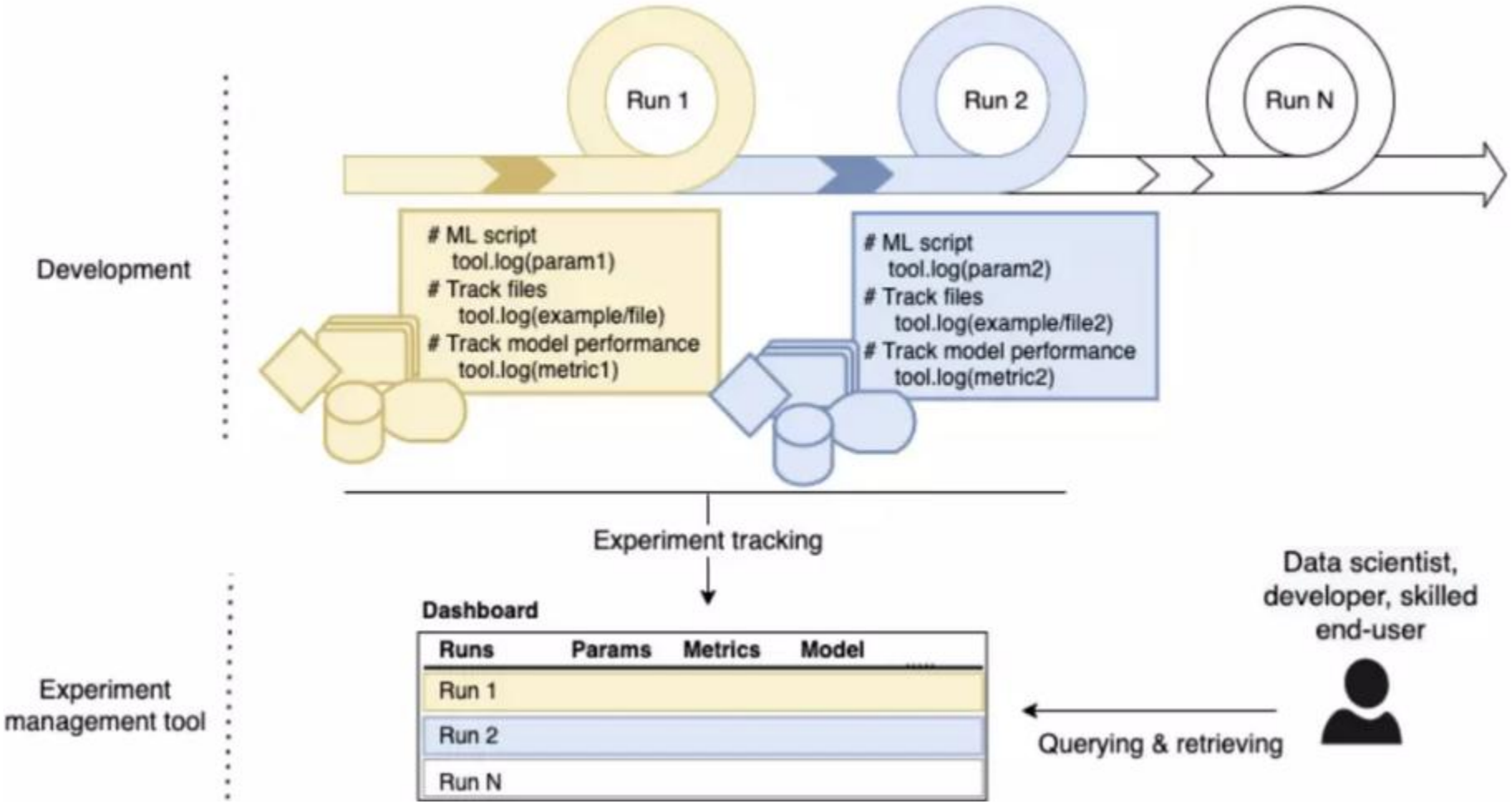
- Flexibility and scalability for real-time predictions, and batch predictions.
- Integration on local serving or cloud platforms.

Collaboration and Knowledge Sharing

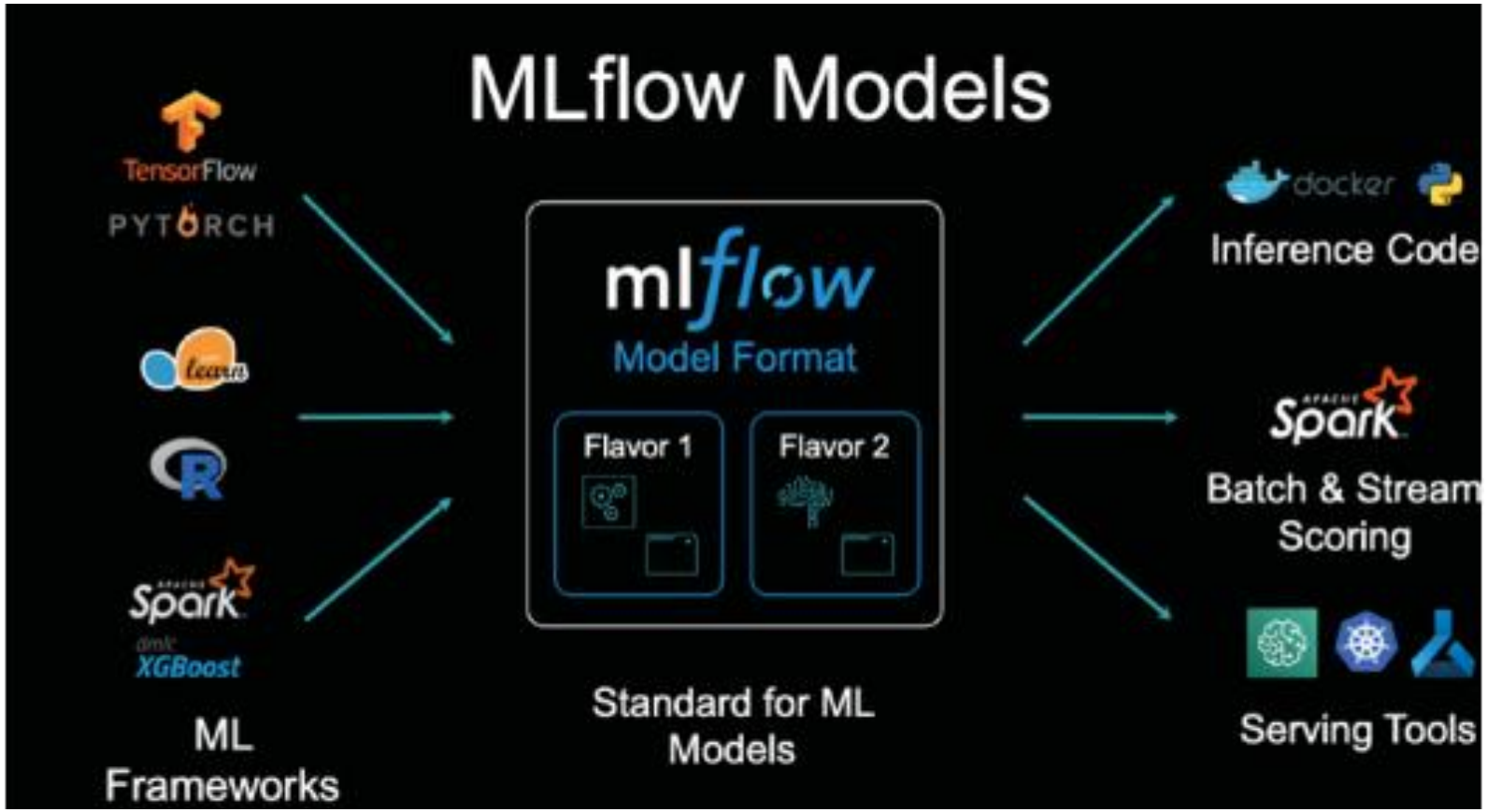
- Visualizations to facilitate communication even for non-technical stakeholders.
- Teams can easily share experiments links, review and discuss.
- Auditors can verify and understand the results.

01: Model Development

What do we need in practice ?

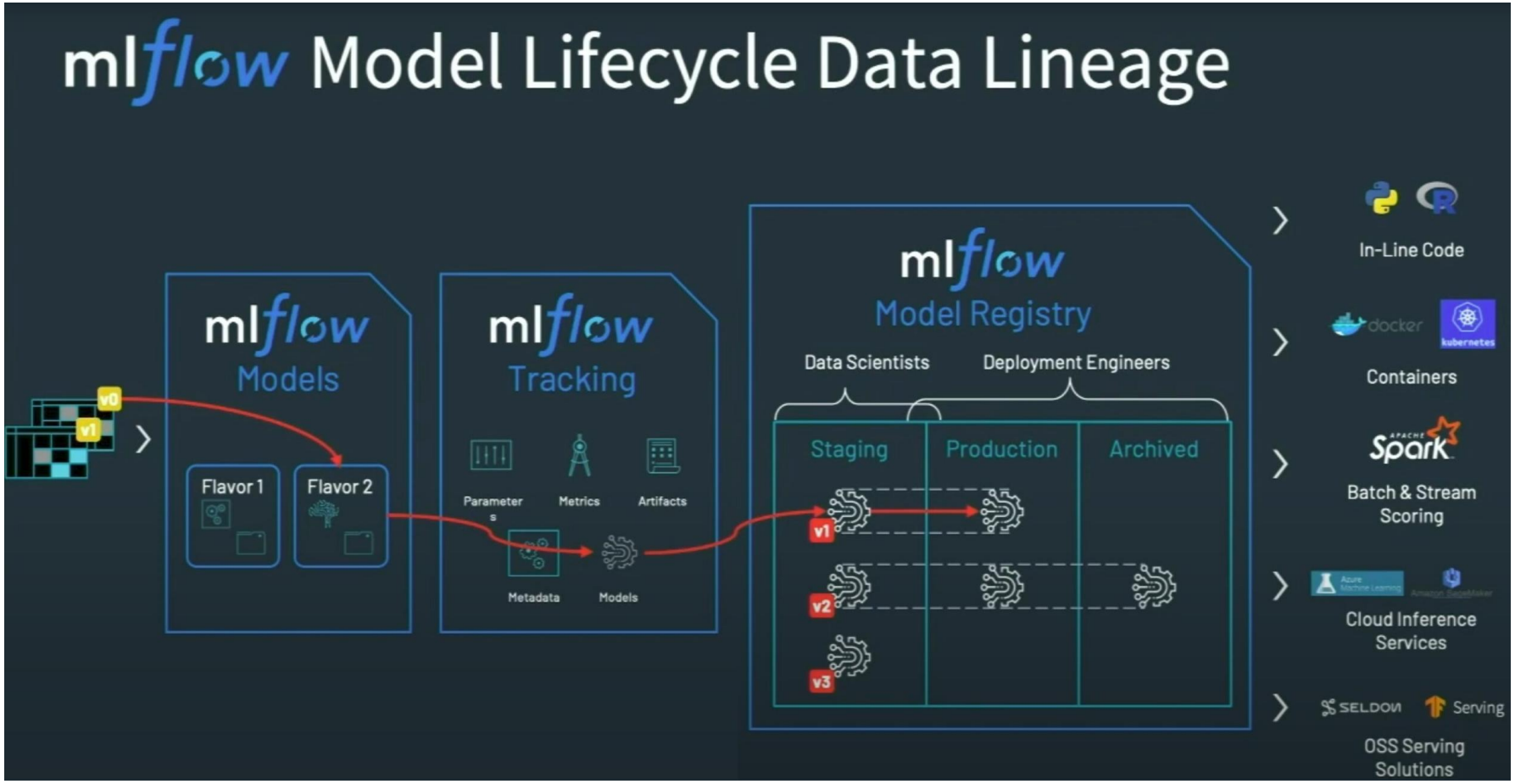


01: Model Development



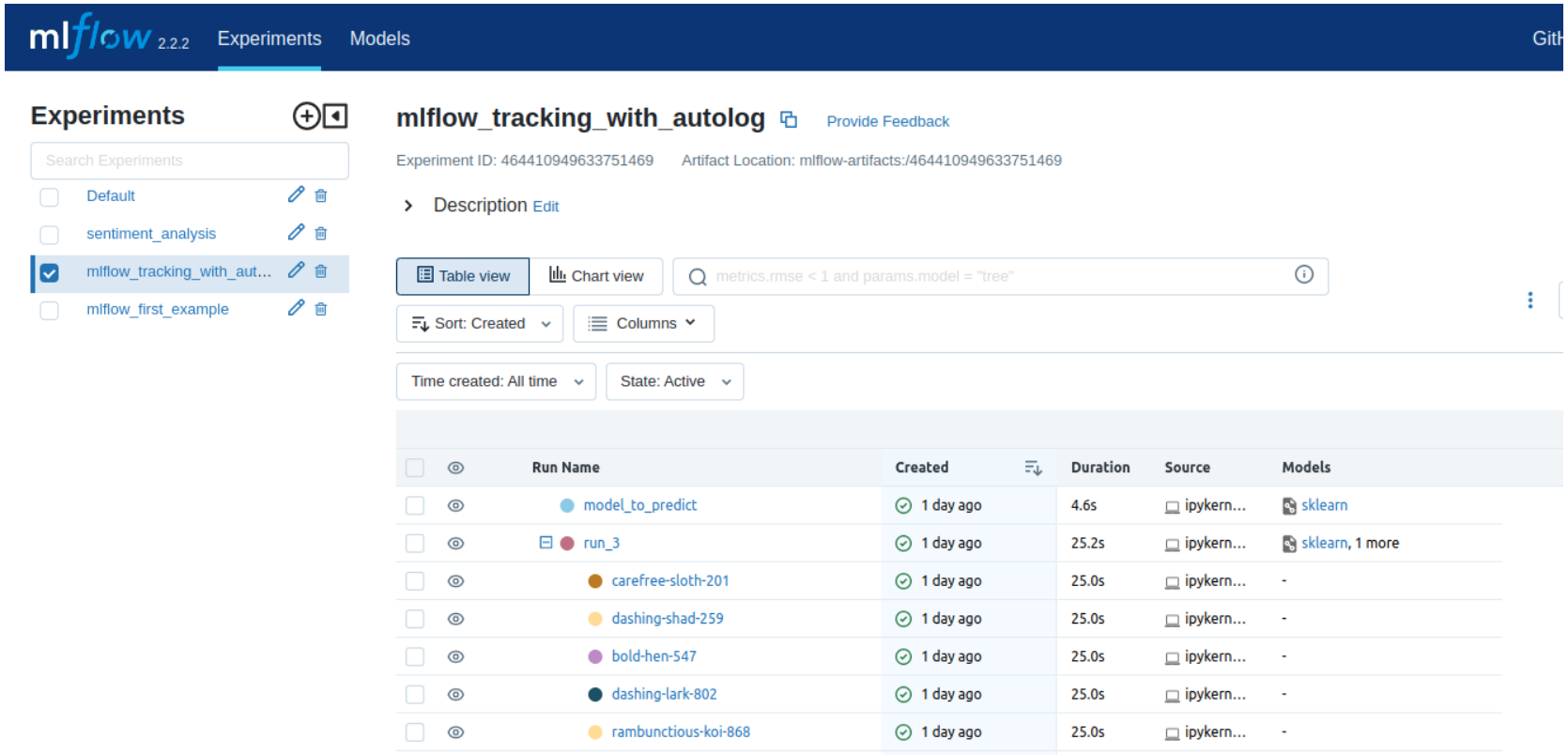
- Power of abstraction.
- It packages and modularizes the models
- UI for easier management of all MLFlow key features.

01: Model Development



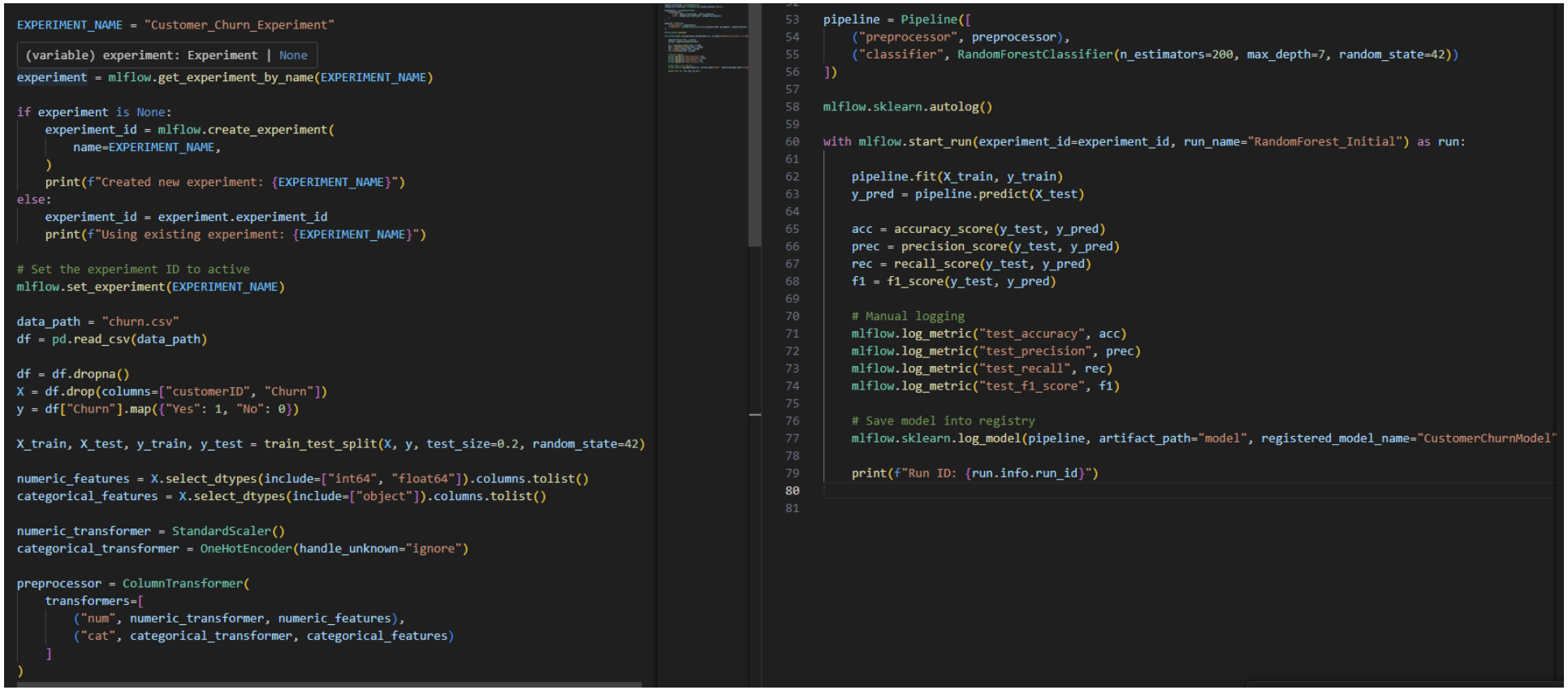
01: Model Development

- **Creating experiments:** We can group our models and relevant metrics. Each experiment can have several runs.



01: Model Development

- **Creating experiments:** We can group our models and relevant metrics. Each experiment can have several runs.



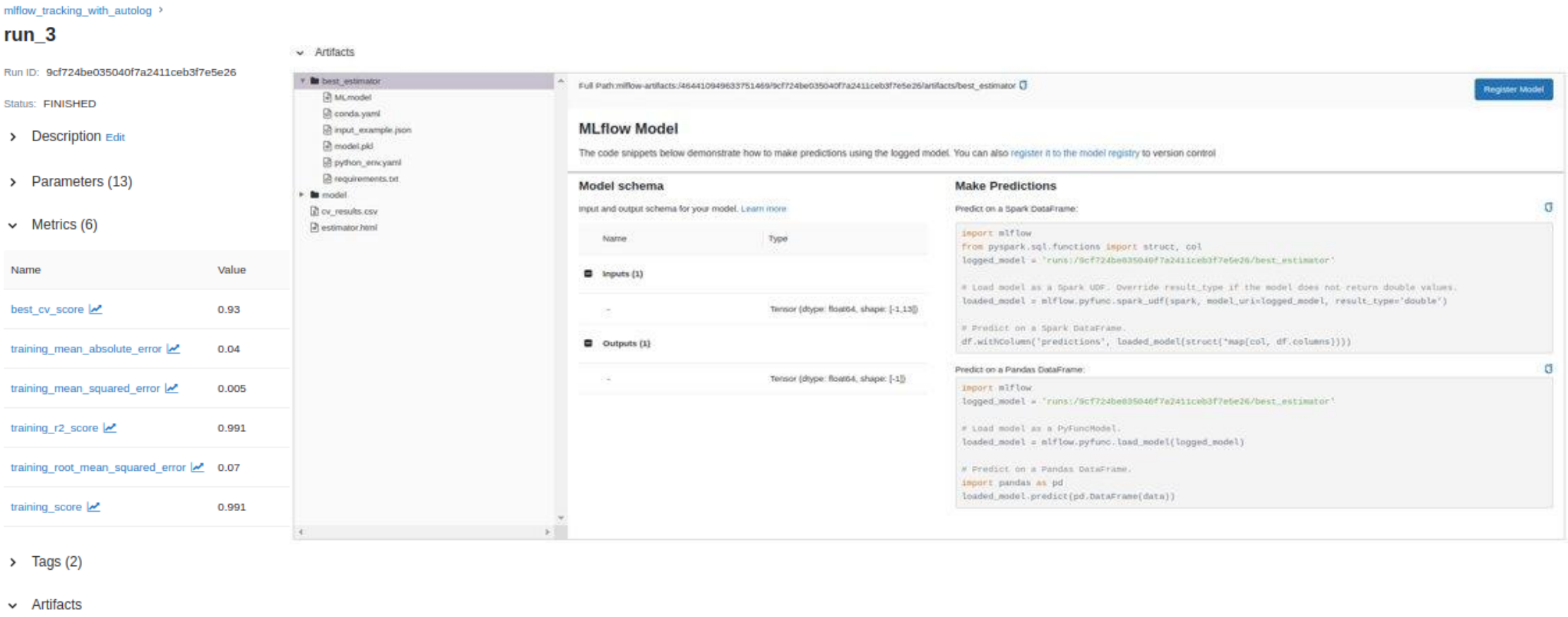
```
EXPERIMENT_NAME = "Customer_Churn_Experiment"
(variable) experiment: Experiment | None
experiment = mlflow.get_experiment_by_name(EXPERIMENT_NAME)

if experiment is None:
    experiment_id = mlflow.create_experiment(
        name=EXPERIMENT_NAME,
    )
    print(f"Created new experiment: {EXPERIMENT_NAME}")
else:
    experiment_id = experiment.experiment_id
    print(f"Using existing experiment: {EXPERIMENT_NAME}")

# Set the experiment ID to active
mlflow.set_experiment(EXPERIMENT_NAME)

data_path = "churn.csv"
df = pd.read_csv(data_path)

df = df.dropna()
X = df.drop(columns=["customerID", "Churn"])
y = df["Churn"].map({"Yes": 1, "No": 0})

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

numeric_features = X.select_dtypes(include=["int64", "float64"]).columns.tolist()
categorical_features = X.select_dtypes(include=["object"]).columns.tolist()

numeric_transformer = StandardScaler()
categorical_transformer = OneHotEncoder(handle_unknown="ignore")

preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features)
    ]
)
```

```
pipeline = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", RandomForestClassifier(n_estimators=200, max_depth=7, random_state=42))
])

mlflow.sklearn.autolog()

with mlflow.start_run(experiment_id=experiment_id, run_name="RandomForest_Initial") as run:

    pipeline.fit(X_train, y_train)
    y_pred = pipeline.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    # Manual logging
    mlflow.log_metric("test_accuracy", acc)
    mlflow.log_metric("test_precision", prec)
    mlflow.log_metric("test_recall", rec)
    mlflow.log_metric("test_f1_score", f1)

    # Save model into registry
    mlflow.sklearn.log_model(pipeline, artifact_path="model", registered_model_name="CustomerChurnModel"

    print(f"Run ID: {run.info.run_id}")
```

- It is easier to **keep track of each run** of our model.
- We can **reproduce the results** since the parameters are saved there.

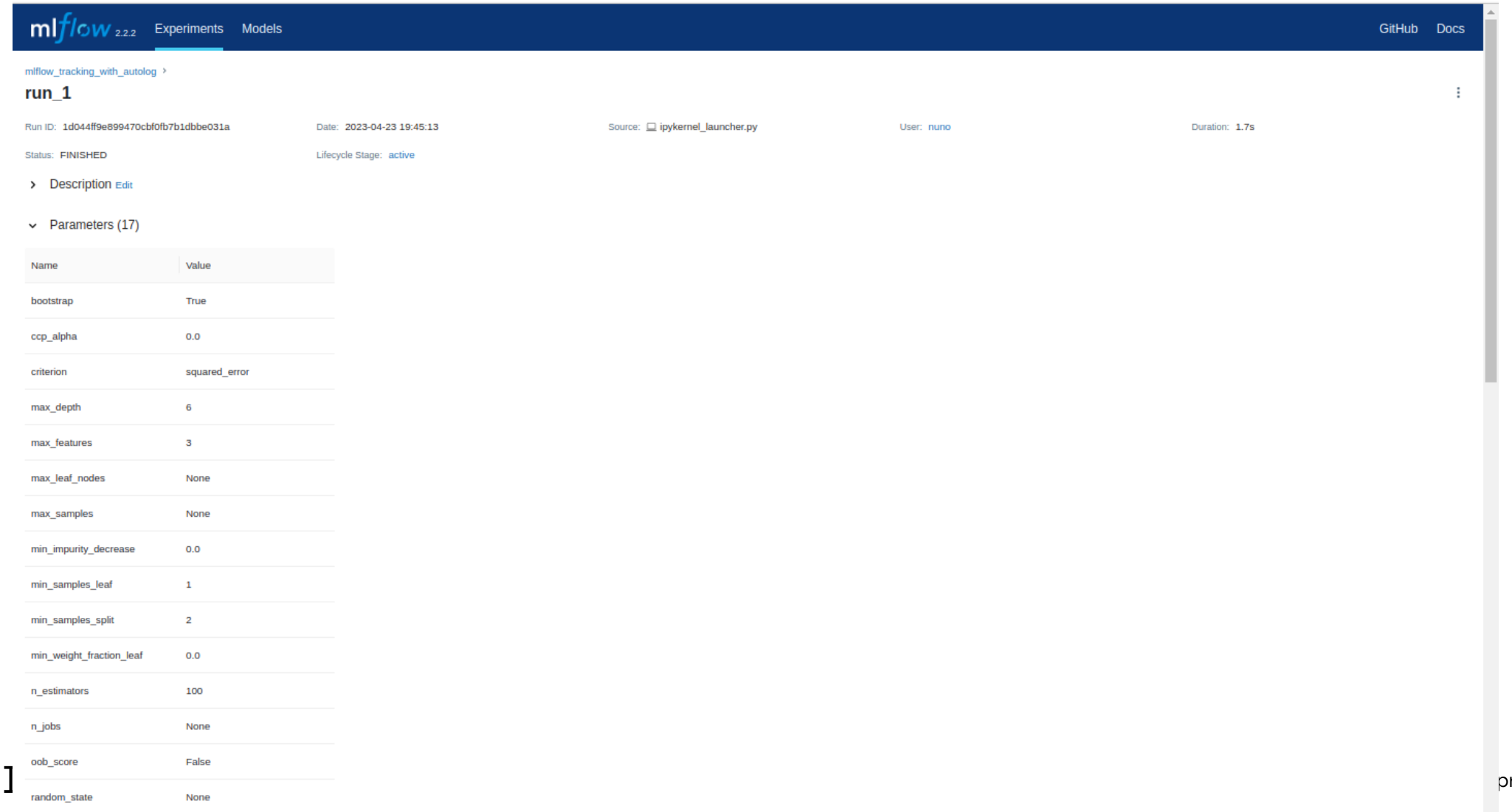
01: Model Development

- **Model and metrics logging:** We can save a model in a modular form and log all of the metrics related to the model run (training, validation and testing). For each run you can decide which metrics you want to save, plots, graphs, confusion matrices, artifacts.



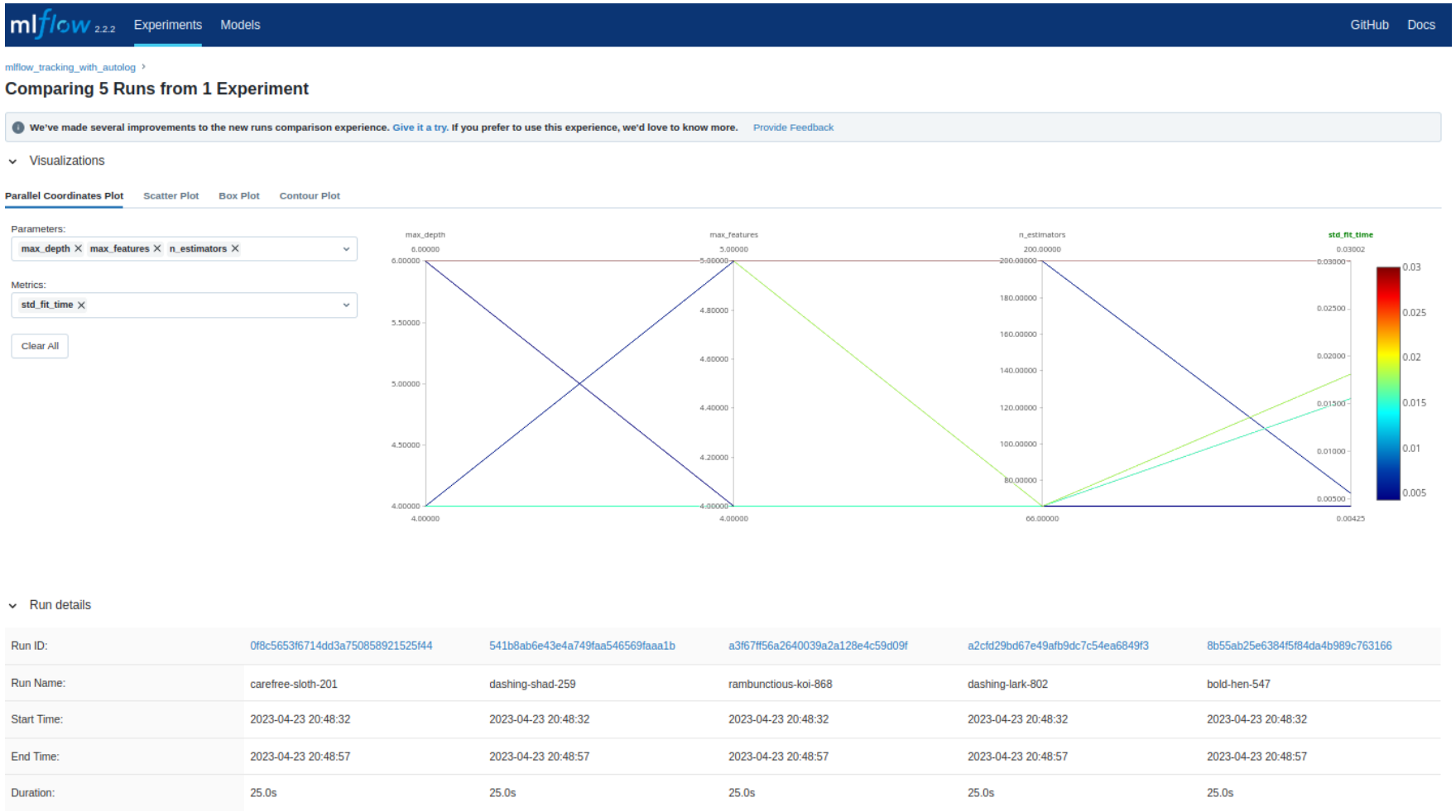
01: Model Development

- **Model and metrics logging:** We can save a model in a modular form and log all of the metrics related to the model run (training, validation and testing). For each run you can decide which metrics you want to save, plots, graphs, confusion matrices and artifacts.



01: Model Development

- **Comparing model metrics:** With MLFlow you can compare different models and their metrics at once, which makes your life easier when it is time to tune the hyperparameters.

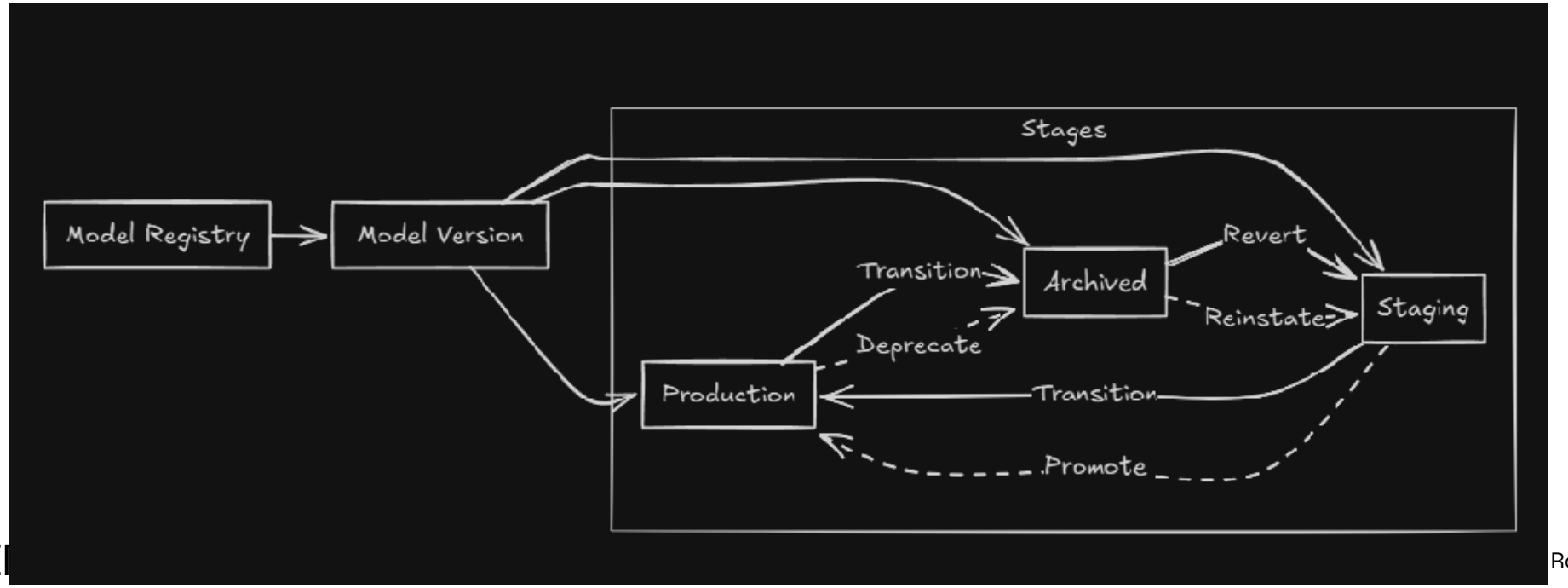


01: Model Development

Aspect	Bad Practice	Good Practice
Experiment tracking	No history	Each run saved with params/metrics
Model saving	Manual overwrite	Logged with versioning
Data preprocessing	In script, not reproducible	Preprocessing in pipeline, saved with model
Evaluation	Manual printouts	Metrics systematically logged

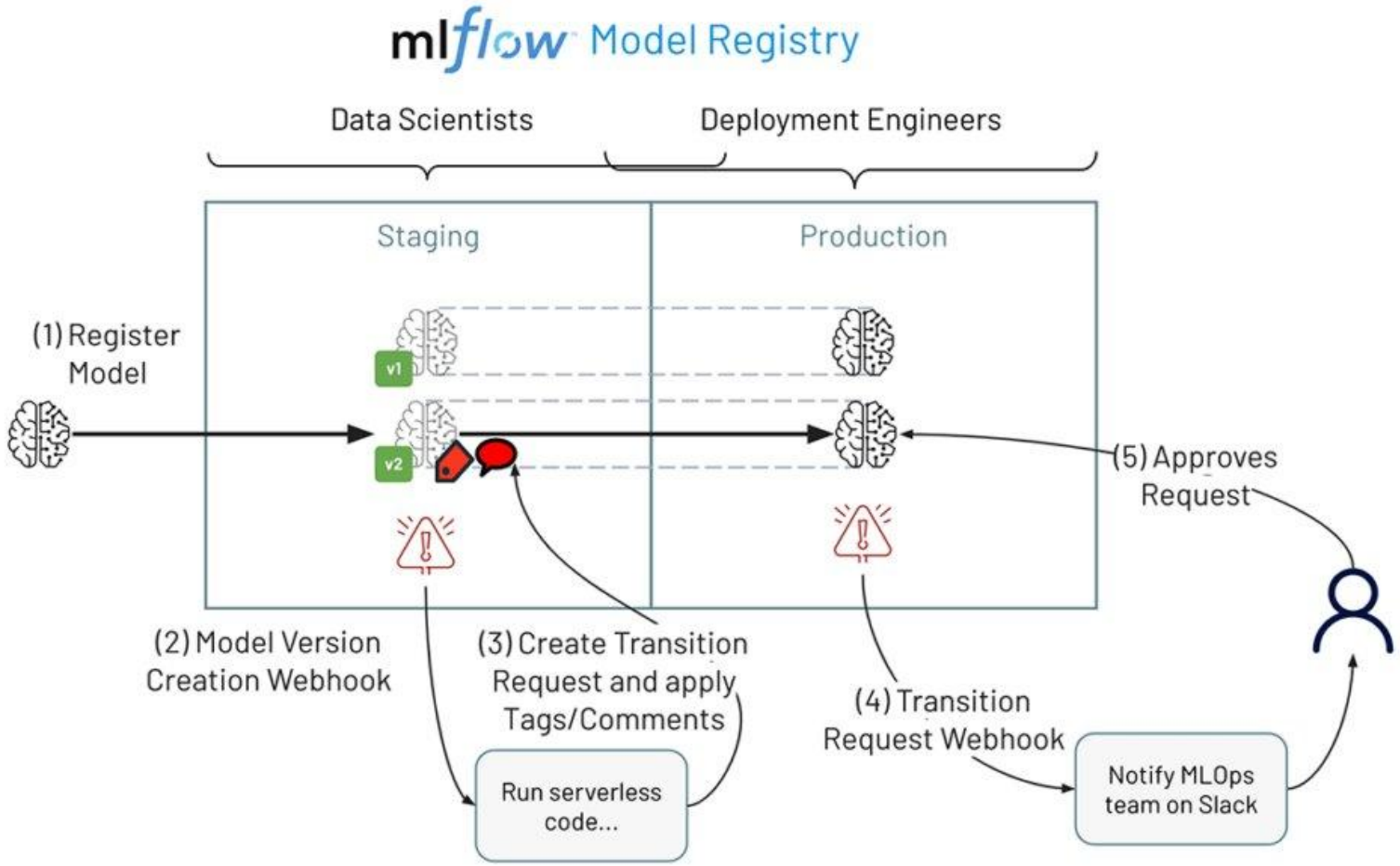
01: Model Development

Aspect	Bad Practice	Good Practice
Experiment tracking	No history	Each run saved with params/metrics
Model saving	Manual overwrite	Logged with versioning
Data preprocessing	In script, not reproducible	Preprocessing in pipeline, saved with model
Evaluation	Manual printouts	Metrics systematically logged



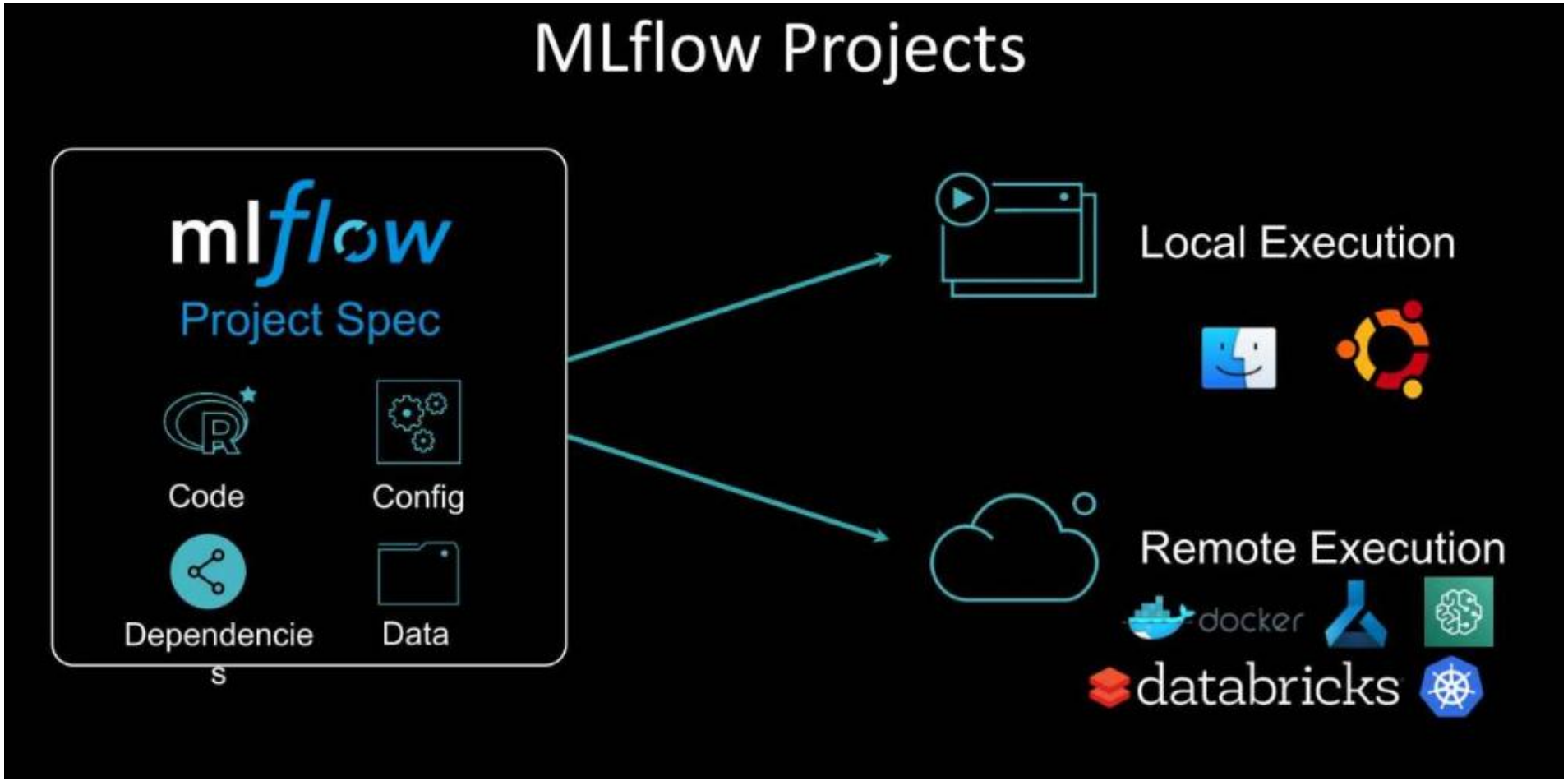
01: Model Development

- **Model registry:** You can register your model and define a particular stage of the development. We can



01: Model Development

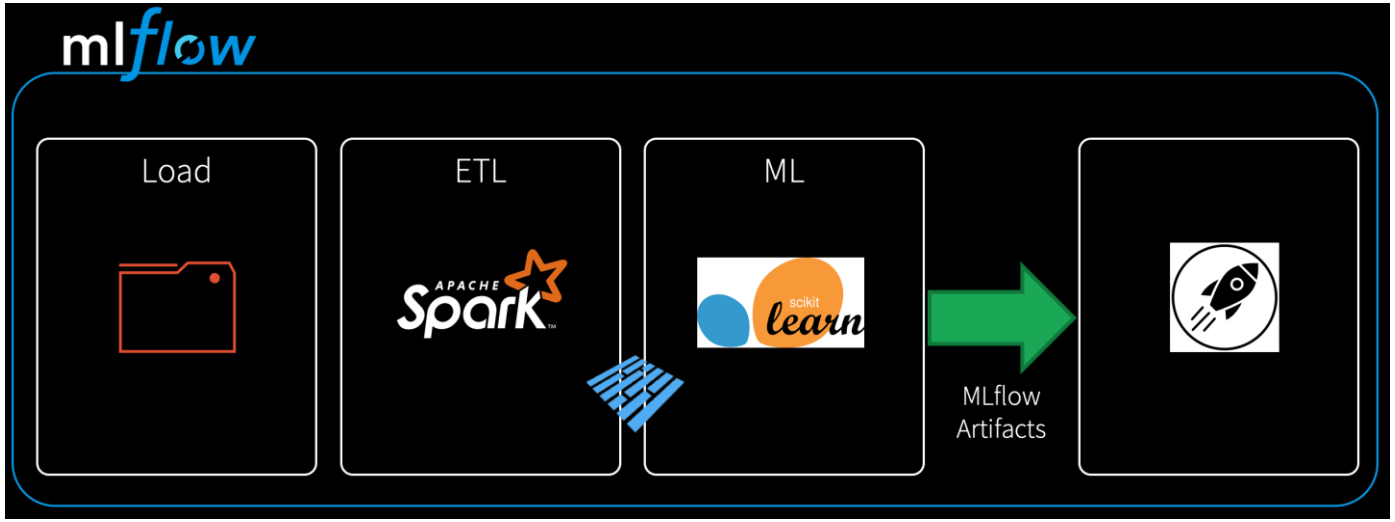
- **Local deployment:** We can deploy on a local server to test model inference. It also can be adapted to work on an hosted server as well.



01: Model Development

MLFlow as our first "maestro"

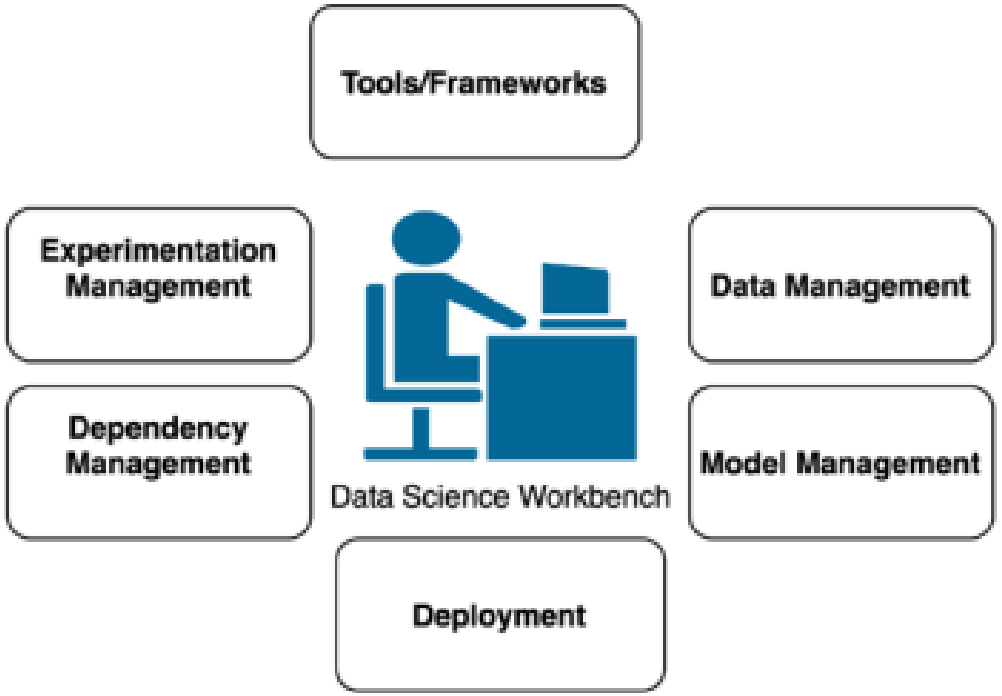
- We can build something really close to **an orchestration tool**.



- **Each component be an experiment**, where we can save all relevant versioned metrics and parameters.
- Possibility of using **Spark to scale to Big Data.**
- **Modular project, capable of dealing with a CD/CI pipeline**, with experiments versioned and well documented.

01: Model Development

What did we accomplish today?



- **Dependency management:** Reproducibility and library conflicts exclusion between different environments.
- **Data management:** Full knowledge of what enters in the model.
- **Model management:** Models organized in a logic of several challengers development vs champion with a certain abstraction level and standardization.
- **Deployment:** Smooth deployment, with agnostic model production.
- **Experimentation Management:** Parameters are stored in the same way -> reproducibility of the experiment.